A DEEP LEARNING FRAMEWORK FOR MULTILINGUAL MEDICINAL PLANT IDENTIFICATION AND INFORMATION RETRIEVAL SYSTEM

A PROJECT REPORT

Submitted by

KOWSIK S	**2022103506**
SOURISH CIDAMBARAM B G	**2022103501**
SELVA SUBRAMANIAN A	**2022103504**
NAGARJUN M	**2022103302**

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY: CHENNAI - 600 025

APRIL 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**A DEEP LEARNING FRAMEWORK FOR MULTILINGUAL MEDICINAL PLANT IDENTIFICATION AND INFORMATION SYSTEM "** is the bonafide work of "**Kowsik S (2022103506) , Sourish Cidambaram B G (2022103501), Selva Subramanian A (2022103504)** and **Nagarjun M (2022103302)**" who carried out the project work under my supervision.

Dr. V . Mary Anitha Rajam

HEAD OF THE DEPARTMENT

Department of Computer Science and Engineering ,

Anna University , Chennai – 600 025.

Dr. R . Baskaran

SUPERVISOR

Professor

Department of Computer Science and Engineering ,

Anna University , Chennai – 600 025.

ACKNOWLEDGEMENT

We express our deep gratitude to our guide, **Dr. R. Baskaran** , Professor, Department of Computer Science and Engineering, for guiding us through every phase of the project. We appreciate his thoroughness, tolerance and ability to share his knowledge with us. We would also like to thank him for his kind support and for providing necessary facilities to carry out the work.

We are extremely grateful to **Dr. V. Mary Anita Rajam**, Professor & Head of the Department, Department of Computer Science and Engineering, Anna University, Chennai – 25, for extending the facilities of the Department towards our project and for her unstinting support.

Our express our thanks to the panel of reviewers **Dr. P. Geetha**, Professor, Department of Computer Science and Engineering, **Dr. S. Sudha,** Professor, Department of Computer Science and Engineering, for their valuable suggestions and critical reviews throughout the course of our project.

We express our thanks to all other teaching and non-teaching staff who helped us in one way or other for the successful completion of the project. We would also like to thank our parents, family and friends for their indirect contribution in the successful completion of this project.

KOWSIK S SOURISH CIDAMBARAM B G SELVA SUBRAMANIAN A NAGARJUN M

ABSTRACT

Medicinal plants have constituted the foundation of traditional healthcare systems such as Siddha, Ayurveda, and Unani for thousands of years, offering holistic therapeutic solutions rooted in the balance between human physiology and nature. However, a substantial portion of this traditional knowledge remains fragmented across regional manuscripts, palm-leaf records, and oral traditions, limiting its accessibility, standardization, and integration into contemporary digital ecosystems. Furthermore, challenges such as visual similarity among plant species, multilingual complexity of classical medical texts, and the absence of structured, searchable databases hinder effective utilization of medicinal plant knowledge in modern contexts.

This research proposes an AI-Based Multilingual Plant Information System that leverages state-of-the-art advancements in Computer Vision (CV) and Natural Language Processing (NLP) to create an intelligent, accessible, and scalable digital framework for medicinal plant identification and knowledge retrieval. The system employs EfficientNetV2B0, a highly optimized convolutional neural network architecture, to extract discriminative visual features from user-provided plant images and perform accurate species classification against a curated Plant Image Database sourced from the National Institute of Siddha (NIS). This ensures reliable recognition even among morphologically similar species.

To support multilingual interaction, the framework integrates the Indic Translation2 model for high-quality translation and cross-lingual representation learning. User queries, provided in text or speech form, are normalized and translated into a unified representation to enable consistent semantic processing. Automatic Speech Recognition (ASR) and Text-to-Speech (TTS) modules further enhance accessibility, enabling seamless voice-based interaction for users with limited literacy or digital proficiency.

For intelligent information retrieval, the system utilizes transformer-based sentence embedding models such as MiniLM to perform semantic search over a structured Plant Text Database. Unlike conventional keyword-based search mechanisms, this approach captures contextual intent and conceptual similarity, significantly improving retrieval accuracy for complex or natural-language queries. Retrieved knowledge is then processed through Flan-T5, a transformer-based generative language model configured for controlled text generation. By grounding outputs in authenticated Siddha knowledge sources, the system ensures factual consistency, domain alignment, and reduction of hallucinated medical claims.

The proposed framework operates as a multimodal, multilingual, and knowledge-grounded AI ecosystem that bridges traditional medicinal wisdom with modern computational intelligence. Beyond information dissemination, the system contributes to digital preservation, biodiversity awareness, and research facilitation by structuring indigenous medicinal data into a machine-readable format. It promotes sustainable knowledge transfer across generations while supporting education, community healthcare awareness, and evidence-based exploration of plant-derived therapeutics.

By harmonizing ancestral knowledge systems with contemporary AI methodologies, this work demonstrates a scalable model for preserving endangered traditional medical knowledge while ensuring accuracy, inclusivity, and technological relevance in the digital era.

திட்டப்பணிச் சுருக்கம்

மருத்துவ மூலிகைகள் ஆயிரக்கணக்கான ஆண்டுகளாக சித்தா, ஆயுர்வேதம், யூனானி போன்ற பாரம்பரிய மருத்துவ முறைகளின் அடித்தளமாக இருந்து வருகின்றன. இயற்கை மற்றும் மனித உடலியல் சமநிலையை மையமாகக் கொண்ட இந்த மருத்துவ முறைகள் முழுமையான மற்றும் தடுப்பு சிகிச்சை அணுகுமுறைகளை வழங்குகின்றன. எனினும், இவ்வாறான பாரம்பரிய அறிவு பெரும்பாலும் பிராந்திய மொழிகளில் எழுதப்பட்ட palm-leaf (ஓலைச்சுவடி) பதிவுகள், வாய்மொழி மரபுகள் மற்றும் சீரமைக்கப்படாத ஆவணங்களில் சிதறிக் கிடப்பதால், அவற்றின் அணுகல், தரநிலைப்படுத்தல் மற்றும் நவீன டிஜிட்டல் அமைப்புகளுடன் ஒருங்கிணைப்பு கடினமாகிறது. மேலும், தோற்றத்தில் ஒத்த மூலிகைகளை துல்லியமாக அடையாளம் காணுதல், பாரம்பரிய மருத்துவ இலக்கியங்களில் பயன்படுத்தப்படும் சிறப்பு சொற்களைப் புரிதல், மற்றும் அமைப்புடனான தேடல் வசதியின்மை போன்ற சவால்களும் காணப்படுகின்றன.

இந்த ஆய்வு, செயற்கை நுண்ணறிவு (AI) அடிப்படையிலான பன்மொழி மூலிகை தகவல் அமைப்பை முன்வைக்கிறது. இது கணினி பார்வை (Computer Vision) மற்றும் இயற்கை மொழி செயலாக்கம் (Natural Language Processing) ஆகிய நவீன தொழில்நுட்பங்களை ஒருங்கிணைத்து, மூலிகை அடையாளம் காணல் மற்றும் மருத்துவ தகவல் மீட்டெடுப்பு செயல்முறைகளை மேம்படுத்துகிறது. இந்த அமைப்பில் EfficientNet எனப்படும் ஆழ்மட்ட கற்றல் மாடல் பயன்படுத்தப்பட்டு, பயனர் வழங்கும் தாவரப் படங்களில் இருந்து இலை வடிவம், நரம்பியல் அமைப்பு, நிறம், அமைப்பு போன்ற தனித்துவ அம்சங்களை பிரித்தெடுத்து, தேசிய சித்த மருத்துவ நிறுவனம் (NIS) வழங்கிய தாவரப் படத் தரவுத்தளத்துடன் ஒப்பிட்டு துல்லியமான அடையாளத்தை வழங்குகிறது.

பன்மொழி ஆதரவிற்காக Indic Translation2 மாடல் பயன்படுத்தப்படுகிறது. பயனர் வழங்கும் உரை அல்லது குரல் உள்ளீடுகள் முதலில் ஒருங்கிணைந்த மொழி வடிவத்திற்கு மொழிபெயர்க்கப்பட்டு பின்னர் பொருளடக்க அடிப்படையில் செயலாக்கப்படுகின்றன. தன்னியக்க குரல் அங்கீகாரம் (ASR) மற்றும் உரை-குரல் மாற்றம் (TTS) போன்ற வசதிகள் மூலம் குறைந்த எழுத்தறிவு கொண்ட பயனர்களுக்கும் எளிதான அணுகல் வழங்கப்படுகிறது.

தகவல் மீட்டெடுப்பிற்காக MiniLM போன்ற transformer அடிப்படையிலான வாக்கிய embedding மாடல்கள் பயன்படுத்தப்பட்டு, keyword அடிப்படையிலான தேடலை விட பொருள் சார்ந்த (semantic) தேடல் செயல்படுத்தப்படுகிறது. இது பயனர் கேள்வியின் உண்மையான நோக்கத்தை புரிந்து, NIS வழங்கிய தாவர உரைத் தரவுத்தளத்தில் இருந்து பொருத்தமான மருத்துவ தகவலை துல்லியமாக மீட்டெடுக்கிறது. பின்னர் Flan-T5 மாடல் பயன்படுத்தப்பட்டு, பெறப்பட்ட தகவலை கட்டுப்படுத்தப்பட்ட மற்றும் உண்மைத்தன்மை கொண்ட விளக்கமாக உருவாக்குகிறது. இதனால் ஆதாரமற்ற அல்லது தவறான மருத்துவக் குறிப்புகள் உருவாகாமல் பாதுகாக்கப்படுகிறது.

இந்த முன்மொழியப்பட்ட அமைப்பு பன்முக, பன்மொழி மற்றும் அறிவு-அடிப்படையிலான செயற்கை நுண்ணறிவு சூழலை உருவாக்குகிறது. இது பாரம்பரிய மருத்துவ அறிவை டிஜிட்டல் வடிவில் பாதுகாத்து, அடுத்த தலைமுறைகளுக்கு பரிமாற உதவுகிறது. மேலும், ஆராய்ச்சி, கல்வி மற்றும் சமூக சுகாதார விழிப்புணர்வை மேம்படுத்தும் ஒரு நிலையான மற்றும் தொழில்நுட்ப ரீதியாக வலுவான தளமாக செயல்படுகிறது.

இவ்வாறு, பாரம்பரிய மூலிகை மருத்துவ அறிவையும் நவீன செயற்கை நுண்ணறிவு தொழில்நுட்பங்களையும் ஒருங்கிணைப்பதன் மூலம், எதிர்கால தலைமுறைகளுக்கான துல்லியமான, அணுகக்கூடிய மற்றும் நிலையான அறிவு பாதுகாப்பு முறையை இந்த ஆய்வு முன்வைக்கிறது.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT – ENGLISH	**iv**
	ABSTRACT – TAMIL	**vi**
	LIST OF TABLES	**xii**
	LIST OF FIGURES	**xiii**
1.	**INTRODUCTION**	
	1.1 OVERVIEW	1
	1.2 OBJECTIVES	4
2.	**LITERATURE SURVEY**	5
3.	**SYSTEM DESIGN**	
	3.1 SYSTEM ARCHITECTURE	13
	3.2 MULTILINGUAL QUERY PROCESSING AND SEMANTIC RETRIEVAL	17
	3.2.1 FASTTEXT MODEL	19
	3.2.2 WHISPER MODEL	21
	3.2.3 HYBRID TRANSLATION MODEL	22
	3.2.4 PREPROCESSING MODULE	25
	3.2.5 SEMANTIC SEARCH MODEL	26

3.3 IMAGE BASED SEMANTIC FEATURE EXTRACTION AND SEARCH 29
3.3.1 IMAGE PREPROCESSING 32
3.3.2 HYBRID VISION MODEL 34
3.4 GENERATIVE MODEL 38
3.4.1 INTENT PREDICTION 40
3.4.2 PROMPT CONSTRUCTION 41
3.4.3 FLAN – T5 MODEL 43

4. RESULTS AND DISCUSSION
4.1 DATASET DESCRIPTION 46
4.2 IMPLEMENTATION DETAILS 49
4.3 OUTPUT 53
4.4 TEST CASES 55
4.5 PERFORMANCE METRICS 58
5. CONCLUSION AND FUTURE WORKS 65
REFERENCES 66

LIST OF TABLES

TABLE	DESCRIPTION	PAGE NO.
4.1	Test Cases	55
4.5.1	Evaluation Metrics for Whisper Model	61
4.5.2	Evaluation Metrics for Fasttext Model	61

LIST OF FIGURES

FIGURE	DESCRIPTION	PAGE NO.
3.1	System Architecture	16
3.2	Module 1 – Query Processing & Semantic Retrieval	18
3.2.1.1	Pseudo Code for Fasttext Model	20
3.2.2.1	Pseudo Code for Whisper Model	22
3.2.3.1	Pseudo Code for Hybrid Translation Model	24
3.2.5.1	Pseudo Code for Semantic Search Model	28
3.3	Module 2 – Hybrid Vision Model	31
3.3.1.1	Pseudo Code for Image Preprocessing	33
3.3.2.1	Pseudo Code for Hybrid Vision Model	36
3.4	Module 3 – Generative Model	39
3.4.1.1	Pseudo Code for Intent Prediction	41
3.4.2.1	Pseudo Code for Prompt Construction	43
3.4.3.1	Pseudo Code for Flan T-5 Model	44
4.2.1	Folder Structure	49
4.3.1	Home Screen	53
4.3.2	Output for Image Input	54
4.3.3	Output for Text Input	54
4.5.3	Evaluation Metrics for Hybrid Translation Model	62
4.5.4	Evaluation Metrics for Semantic Search Model	62
4.5.5	Graph for Each Models Accuracy Comparison	63
4.5.6	Training Vs Validation Accuracy Curve	64

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Medicinal plants have been an integral part of human civilization for millennia, forming the foundation of healthcare practices across diverse cultures. Long before the advent of modern medicine and synthetic pharmaceuticals, human societies relied on nature's resources to treat diseases, strengthen immunity, and maintain overall well-being. Ancient medical systems such as Siddha, Ayurveda, and Unani extensively employed plant-based remedies, emphasizing holistic health, balance of bodily elements, and preventive care. These traditional systems viewed health as a harmonious interaction between the body, mind, and environment, where medicinal plants played a central role in both treatment and prevention.

In the Indian subcontinent, medicinal plant knowledge is deeply embedded in cultural, spiritual, and literary traditions. The Siddha system, originating in Tamil Nadu, classifies medicinal plants based on elemental composition and physiological effects, reflecting a sophisticated understanding of human–nature balance. Over centuries, this knowledge was preserved through palm-leaf manuscripts, classical texts, and oral traditions maintained by traditional healers and scholars. However, much of this information remains scattered across regional languages and non-digitized sources, limiting accessibility for modern research and public use. Rapid urbanization, biodiversity loss, and declining intergenerational knowledge transfer further threaten the survival of this traditional wisdom.

Technological barriers also contribute significantly to the erosion of medicinal plant knowledge. Existing digital resources are fragmented, largely

monolingual, and lack standardized representations. Accurate identification of medicinal plants remains challenging due to visual similarity among species, while understanding traditional medicinal descriptions requires familiarity with domain-specific terminology and classical languages. These limitations highlight the need for an intelligent, multilingual, and multimodal system capable of accurately identifying plants and retrieving reliable medicinal information.

Recent advancements in Artificial Intelligence (AI) provide powerful tools to address these challenges. In particular, progress in Computer Vision (CV) and Natural Language Processing (NLP) enables automated plant identification, multilingual understanding, and intelligent information retrieval. In the proposed AI-Based Multilingual Plant Information System, deep learning–based Computer Vision models are used to identify plants from images, while NLP models enable multilingual query processing, semantic understanding, and controlled text generation.

For visual plant recognition, the system employs EfficientNet, a convolutional neural network architecture known for its optimal balance between accuracy and computational efficiency. EfficientNet extracts discriminative visual features such as leaf shape, texture, venation, and color patterns, enabling accurate matching of user-provided images with the Plant Image Database curated by the National Institute of Siddha (NIS). This allows reliable plant identification even for visually similar species.

For multilingual text and speech-based interaction, the system integrates the Indic Translation2 model, which enables high-quality translation across multiple Indian and international languages. User queries provided in text or voice form are first normalized and translated into a unified English representation, ensuring consistent downstream processing. Speech inputs are handled using automatic speech recognition, enabling accessibility for users with limited literacy or typing skills.

To retrieve relevant medicinal knowledge from large textual repositories, the system employs a semantic search model based on sentence-level embeddings

generated using transformer architectures such as MiniLM. Unlike keyword-based search, semantic search captures the contextual meaning of user queries and matches them with conceptually relevant plant descriptions stored in the Plant Text Database provided by NIS. This approach significantly improves retrieval accuracy for complex or natural-language queries.

Once relevant information is retrieved, the system uses Flan-T5, a transformer-based generative language model, to generate factually constrained and context-aware plant descriptions. Flan-T5 maps retrieved knowledge into structured prompts and produces clear, readable responses while avoiding unsupported medical claims. This controlled generation approach ensures that the output remains faithful to authenticated Siddha knowledge rather than speculative or hallucinated content.

To enhance inclusivity, the generated responses are translated back into the user's preferred language using the Indic Translation2 model. Additionally, a text-to-speech module converts textual output into audio form, allowing users to receive information in both text and voice formats. This multimodal output significantly improves usability for elderly users, rural communities, and individuals with reading difficulties.

In summary, the proposed AI-Based Multilingual Plant Information System integrates EfficientNet for image-based plant identification, Indic Translation2 for multilingual translation, semantic search models for intelligent knowledge retrieval, and Flan-T5 for controlled text generation, all grounded in authoritative datasets from the National Institute of Siddha (NIS). By combining traditional medicinal knowledge with state-of-the-art AI techniques, the system serves as a robust digital bridge between ancestral wisdom and modern technology. Rather than replacing traditional practices, this approach revitalizes and preserves them, ensuring that medicinal plant knowledge remains accurate, accessible, and relevant for future generations.

1.2 OBJECTIVES

The major objectives of this project work titled multilingual medicinal plant identification and information retrieval system using deep learning techniques is

- To build a multilingual medicinal plant identification system using advanced image recognition techniques for accurate plant detection.
- To develop a comprehensive digital repository containing plant details, medicinally useful parts, traditional remedies, and preparation recipes.
- To design a module that can process Tamil poem images through OCR and NLP , extract plant references from classical texts and map them to relevant medicinal knowledge .
- To ensure that the system contributes to the preservation and democratization of traditional medicinal wisdom by making it easily accessible to users in multiple languages.
- To provide an interactive web-based platform that integrates AI technologies with traditional knowledge, ensuring usability for researchers, practitioners, and the general public

CHAPTER 2
LITERATURE SURVEY

2.1Machine Translation and Multilingual Language Processing :

The study by Wang et al. [1] provides an extensive overview of how large language models are transforming interactive machine translation (IMT). The authors categorize recent developments into adaptive human-in-the-loop workflows, multimodal assistance, controllable decoding, and low-resource adaptation. Their review highlights that LLMs significantly enhance contextual understanding and allow interactive correction through user feedback, improving translation refinement in real time. A key strength is the structured taxonomy that links model capabilities to practical IMT needs. However, the survey also notes that most IMT systems are evaluated on synthetic interaction settings rather than real human usage, limiting conclusions about usability, latency, and long-term user adaptation.

The work of Liang et al. [2] introduces a LoRA-based fine-tuning approach tailored for low-resource languages, enabling efficient specialization of large language models while updating only a small fraction of their parameters. Their method applies low-rank adaptation layers to selectively modify attention and feed-forward modules, achieving notable improvements in BLEU and chrF scores even with minimal bilingual data. This makes the method highly suitable for languages lacking extensive corpora. Despite its efficiency, the authors acknowledge that performance gains are sensitive to the quality of the limited training data, and complex morphology or domain shift can still challenge the system’s robustness.

2.2 Speech Recognition and Speech Embedding Techniques:

The investigation by Porjazovski et al. [3] systematically compares a diverse set of speech-embedding techniques designed to convert raw audio into stable, fixed-length representations for downstream tasks. Their evaluation spans handcrafted features, self-supervised encoders, and task-specific pooling strategies, assessing each using consistent intrinsic and extrinsic benchmarks. Findings show that modern self-supervised speech encoders outperform traditional pipelines across classification and retrieval tasks, though the optimal pooling method depends heavily on the application. A limitation identified is that the benchmarks primarily use clean, well-resourced languages, making it unclear how effectively these embeddings generalize to noisy, low-resource, or field-recorded speech scenarios.

2.3 Semantic Retrieval and Embedding-Based Search Systems:

In their study, Weckmüller et al. [4] propose a multilingual geo-semantic search framework that leverages text embeddings to enhance the analysis of urban geo-textual datasets such as social media posts and volunteered geographic information. The system precomputes multilingual sentence embeddings, aggregates them spatially, and visualizes semantic patterns to support urban research tasks like hotspot detection or place-based sentiment analysis. Their results show improved cross-lingual retrieval and more meaningful spatial patterns compared to keyword-based methods. However, the authors note that embedding bias, inaccurate geo-tags, and uneven data distribution can lead to distorted interpretations, necessitating careful domain validation.

The survey conducted by Hong and Park [5] examines how LLMs are increasingly being used to perform semantic joins linking heterogeneous text records based on meaning rather than exact attribute matching. They discuss prompting strategies, hybrid retrieval-generation approaches, and verification

mechanisms that help LLMs infer latent equivalences between data entries. The review highlights that LLM-based joins provide flexibility and robustness to paraphrasing, outperforming rigid rule-based schema matching. Nevertheless, the authors point out that hallucinations and non-deterministic outputs remain obstacles, underscoring the need for constraint-guided verification and scalable evaluation frameworks for large datasets.

The comprehensive review by Zouine et al. [6] synthesizes findings on the antibacterial properties of plant extracts, evaluating how extraction methods, solvents, plant parts, and phytochemical profiles influence antibacterial potency. Their analysis details mechanisms such as membrane disruption, efflux pump inhibition, and quorum-sensing interference, offering insight into why certain extracts exhibit stronger activity. While the review highlights many promising plant-derived compounds, the authors stress that most studies remain limited to in vitro assays, with scarce in vivo validation or toxicity profiling. This gap restricts clinical translation, emphasizing the need for standardized protocols and pharmacological evaluation.

The systematic review conducted by Sharrab et al. [7] offers a comprehensive examination of how transformer-based deep learning architectures have reshaped modern speech recognition systems. The authors analyze major model families—including encoder–decoder transformers, conformer hybrids, self-supervised pretraining frameworks, and large-scale multilingual speech models—highlighting how attention mechanisms, contextual modeling, and large corpora contribute to improved robustness and accuracy. Their survey identifies key innovations such as efficient attention variants, adaptive pretraining objectives, and multimodal extensions that enhance performance under noisy or low-resource conditions. A notable contribution of the paper is its discussion of practical deployment challenges, including computational overhead, latency constraints, and ethical considerations related to bias in speech datasets. Despite the progress reported, the authors point out that real-time processing for edge devices, improved handling of code-switching

speech, and reducing dependency on massive annotated corpora remain open research problems requiring future exploration.

2.4 EfficientNet-Based Vision Models for Image Classification:

The study by Rahman et al [8]. presents an EfficientNet-based deep learning approach for classifying diabetic foot ulcer images into ischemia and infection categories, aiming to support early and automated clinical diagnosis. The proposed framework leverages transfer learning and data augmentation to improve feature discrimination while maintaining computational efficiency. A key strength of the work lies in its lightweight architecture, which enables practical deployment in resource-constrained healthcare environments. However, the study relies solely on RGB image data and evaluates performance on controlled datasets, limiting insights into robustness under real-world clinical conditions involving diverse imaging setups and patient variability.

The study by Wencheanteh [9] introduces a hybrid crop disease detection framework that combines EfficientNet for local feature extraction with a group-wise transformer to model global contextual relationships in agricultural images. This design effectively balances fine-grained texture learning with long-range dependency modeling, leading to improved classification accuracy across multiple crop disease datasets. A notable contribution is the reduction of transformer computational overhead through group-wise attention mechanisms. Nevertheless, the approach depends heavily on large annotated datasets and may face limitations when applied to rare diseases or low-resource agricultural scenarios.

The study by Prashant Johr [10] proposes EMANet, a deep learning-based framework for ancient text detection that integrates an enhanced EfficientNet backbone with multidimensional scale fusion to address variability in script size and document degradation. The model demonstrates strong robustness to noise, uneven illumination, and complex backgrounds commonly found in historical manuscripts. A key strength of the work is its ability to preserve semantic

consistency across multiple feature scales. However, the evaluation is largely limited to specific scripts and datasets, leaving questions about generalization to unseen languages, severely damaged texts, and large-scale digitization pipelines unanswered.

The study by Preetha et al[11]. investigates a fine-tuned EfficientNet-B4 convolutional neural network for automated brain tumor detection from MRI images. By optimizing hyperparameters and employing transfer learning, the model achieves high classification accuracy and outperforms several existing CNN architectures. A key strength of the work is the comprehensive ablation and comparative analysis that validates architectural and training choices. However, the approach focuses only on binary classification and does not provide tumor localization or segmentation, limiting interpretability and clinical usefulness in treatment planning.

The study by Saba et al.[12] proposes a synergistic diagnostic framework that combines EfficientNet-based feature extraction with the NSGA-II multi-objective genetic algorithm for optimal feature selection in colon cancer detection. The integration effectively reduces feature redundancy and improves diagnostic accuracy across multiple histopathological datasets. A notable strength is the explicit emphasis on balancing accuracy and computational efficiency. However, the use of evolutionary optimization increases computational complexity and training time, potentially limiting scalability and real-time clinical deployment.

The study by Zhang et al[13]. introduces EfficientNet-EA, a visual location recognition framework that enhances EfficientNet with an Efficient Feature Aggregation layer and employs multi-similarity metric learning for robust scene matching. The model demonstrates strong resilience to environmental variations such as lighting changes, weather conditions, viewpoint shifts, and partial occlusions. A key strength is the generation of compact yet discriminative global descriptors suitable for real-time applications. However, the system's performance can still degrade in highly dynamic environments, and the reliance

on static image inputs limits exploitation of temporal information available in video-based localization.

2.5 Generative Artificial Intelligence and Intelligent Decision Systems:

The study by Tosi [14] evaluates the effectiveness of generative AI in conducting systematic literature reviews by comparing a GPT-4–powered tool with a human-led SLR in the domain of Big Data research. The findings demonstrate that while generative AI significantly accelerates literature retrieval and favors highly cited studies, it suffers from limited recall, hallucinated references, and insufficient critical synthesis. A major strength of the work is its empirical comparison using precision, recall, and overlap analysis, clearly exposing the trade-offs between automation and scholarly rigor. However, the study concludes that fully automated SLRs remain unreliable, emphasizing the necessity of hybrid AI–human review frameworks for maintaining validity and depth.

The study by Teoh et al. [15] investigates the integration of generative AI with YOLO-based object detection to improve flood disaster response in Malaysia. By generating synthetic flood images and combining them with real drone data, the model enhances detection performance during emergency response phases. A key strength of the study is its innovative use of generative data to overcome the scarcity of real flood imagery and its strong empirical results on detection accuracy. However, the work is limited by potential domain mismatch between synthetic and real-world flood scenarios, raising concerns about large-scale generalization and operational deployment.

The study by Rahman et al [16] . proposes MASQRAD, a multiagent actor–critic generative AI framework designed to resolve ambiguous user queries and generate actionable insights. The system leverages cooperative agents to refine queries, validate generated code, and interpret results, significantly improving accuracy and interpretability. A major strength is its structured multiagent design, which mitigates hallucinations and enhances analytical reliability. However, the

increased system complexity and coordination requirements introduce scalability and computational challenges, particularly for real-time, large-scale deployments.

The study by Guan [17] presents a generative AI–driven web-based platform for real-time flight price prediction using historical and live data streams. By combining generative models with deep learning architectures, the system captures complex pricing dynamics and adapts to volatile market conditions. A key strength is its emphasis on real-time responsiveness and integration with web engineering frameworks. However, the approach remains sensitive to abrupt market changes and external shocks, which may limit prediction reliability during extreme or unforeseen events.

The study by Ankalaki et al. [18] provides a comprehensive survey of cyber-attack prediction techniques, tracing the evolution from traditional machine learning to deep learning and generative AI. The review highlights how generative models enable threat simulation, anomaly detection, and attack pattern generation. A major contribution lies in its structured comparison of AI paradigms and emphasis on explainability in cybersecurity. However, the survey identifies persistent challenges related to adversarial robustness, ethical risks, and the black-box nature of generative models, limiting their standalone deployment in security-critical systems.

The study by Deng et al.[19] conducts a systematic literature review examining the role of generative AI in intrusion detection systems for IoT environments. The review categorizes GenAI applications into data augmentation, reconstruction, and adversarial attack generation, highlighting their effectiveness in addressing data imbalance and evolving threats. A key strength is the comprehensive taxonomy and identification of research trends across GAN and transformer-based models. However, the study reveals a lack of standardized datasets and real-world evaluations, limiting the generalizability of current findings.

The study by Kim et al.[20] presents a comprehensive taxonomy and survey of UI/UX considerations for generative AI systems across text, image, audio, and multimodal domains. The work emphasizes aligning AI functionality with human-centered design principles to improve usability and trust. A key strength is its structured modality-based taxonomy that bridges technical architectures with interaction design. However, the study largely remains conceptual, with limited empirical validation, highlighting the need for user-driven evaluation methodologies in real-world generative AI systems.

.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE:

The proposed AI-Based Multilingual Plant Information System is designed to automate and streamline the process of multilingual medicinal plant identification, semantic retrieval, and structured knowledge delivery using advanced artificial intelligence techniques. Unlike traditional static plant information systems, this architecture integrates multilingual Natural Language Processing (NLP), speech recognition, translation models, and hybrid search strategies into a unified end-to-end pipeline. The system ensures accessibility, linguistic inclusivity, and accurate retrieval of authenticated medicinal plant information from authoritative Siddha datasets.

The system is composed of multiple interconnected modules to guarantee efficient processing, identification, retrieval, and presentation of plant-related knowledge.

1. Multimodal Input Processing: The multimodal input module enables users to submit queries either in text or audio form across supported Indian languages. This dual-input mechanism enhances usability, especially for users with limited typing proficiency or literacy constraints.For text-based queries, the system directly forwards the input to the language detection pipeline. For audio-based queries, the spoken input is first processed through an Automatic Speech Recognition (ASR) module.The ASR component utilizes Whisper Large v3 Turbo, a state-of-the-art multilingual speech recognition model capable of highly robust transcription across diverse Indian languages. This ensures accurate conversion of spoken queries into text while preserving linguistic integrity for downstream processing.

2. Language Detection and Preprocessing : The language detection module identifies the source language of user queries using the fastText language identification model. This model is optimized for short and noisy real-world queries and provides reliable language codes for multilingual routing.Once detected, the system performs preprocessing and normalization, including:

- Removal of punctuation and special characters
- Unicode normalization
- Standardization of numerals
- Script variation handling
- Transliteration to Latin representation (when required for Indic languages)
- Lightweight tokenization and stemming
- This stage ensures consistency across multilingual inputs and enhances matching accuracy in later retrieval steps.

3. Multilingual Translation and Unified Indexing: To maintain a consistent retrieval strategy, all processed queries are translated into English, which serves as the system's pivot indexing language. Translation is performed using a hybrid approach combining IndicTrans2 + NLLB. These models are specifically chosen for their strong performance on Indian and low-resource languages. By translating queries into a unified language, the system simplifies database indexing, reduces cross-language ambiguity, and ensures retrieval consistency across multilingual users.

4. Hybrid Candidate Matching and Semantic Retrieval: The core intelligence of the system lies in its hybrid retrieval module, which combines multiple search strategies to improve plant identification accuracy. It performs exact and transliteration matching for precise name comparisons, applies fuzzy matching to handle spelling variations, and uses dense semantic search with Sentence-Transformers embeddings to measure contextual similarity between queries and plant descriptions. The final ranking score is generated by combining fuzzy matching scores, semantic similarity scores, and language-aware ranking boosts, ensuring high recall and precision in retrieving relevant plant information.

5. Knowledge Retrieval and Intelligent Response Assembly:

Once the best-matching plant candidate is identified, the response assembly module retrieves verified plant attributes from the authoritative dataset provided by the National Institute of Siddha. Depending on user intent, the system can:

- Return a specific attribute (e.g., Tamil name, botanical name, medicinal use)
- Generate a structured descriptive response
- Provide comprehensive plant information
- This module ensures flexibility while maintaining precision and authenticity in knowledge delivery.

6. Web Application Interface and Output Presentation :

The final output is delivered through a user-friendly mobile application interface that includes separate tabs for text and audio input, a recognized text display for verification, a structured answer section, and real-time response rendering. This interface ensures intuitive access to multilingual medicinal plant knowledge, improving usability, inclusivity, and accessibility for diverse user groups.

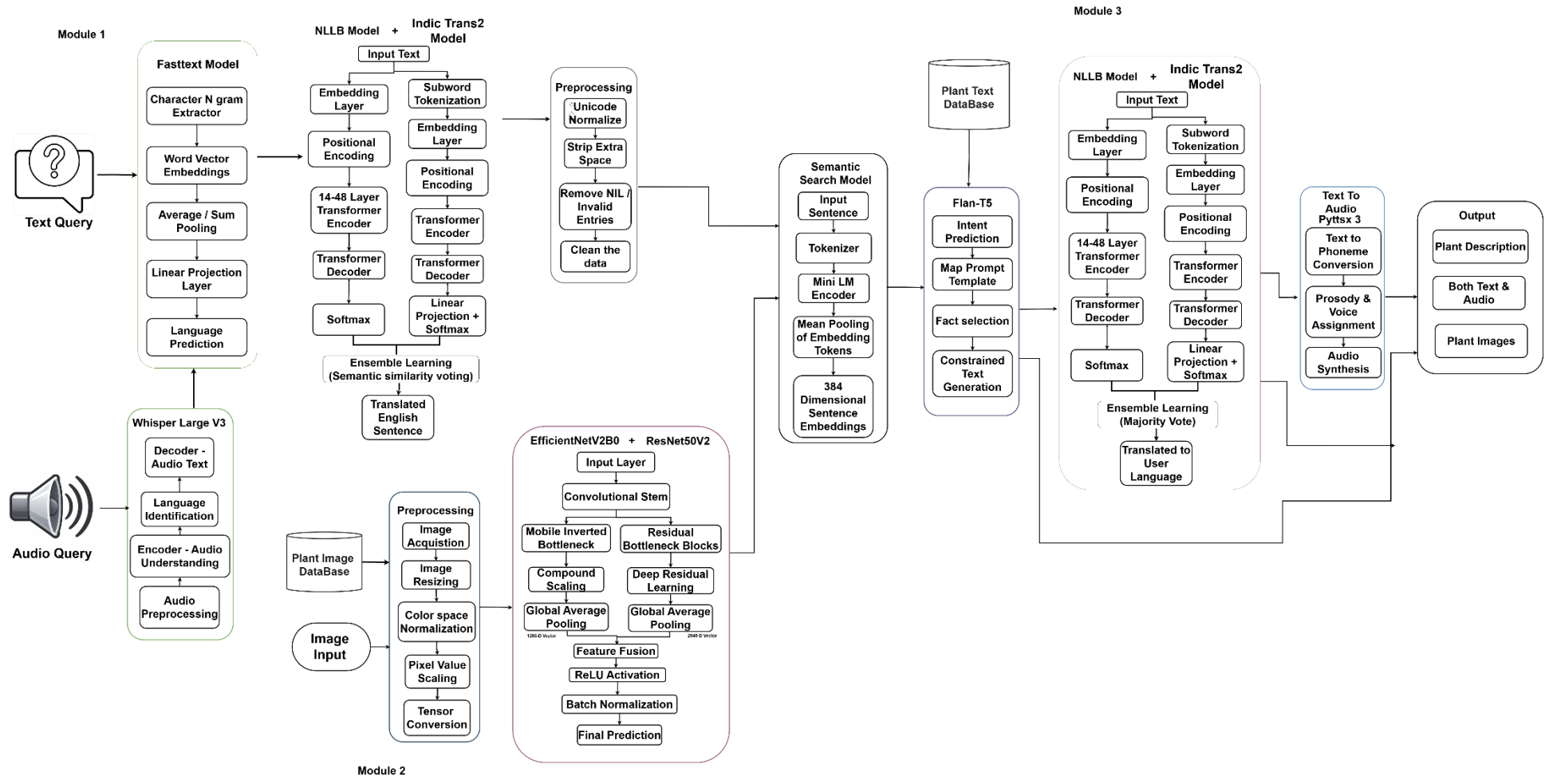

Figure 3.1 System architecture

3.2 MULTILINGUAL QUERY PROCESSING AND SEMANTIC RETRIEVAL

Multilingual query processing and semantic retrieval functions as the core linguistic and semantic processing layer of the AI-Based Multilingual Plant Information System, enabling users to submit medicinal plant queries in any Indian language through text or speech and receive accurate, structured information. The pipeline begins with automatic language detection using fastText, which utilizes character-level n-grams to robustly identify short and noisy multilingual inputs. The query then undergoes preprocessing, including Unicode normalization (NFC), noise removal, transliteration, and text standardization. For audio queries, speech is transcribed using the Whisper Large v3 Turbo model before entering the same pipeline. To ensure universal language support, a hybrid translation mechanism combining NLLB (No Language Left Behind) and IndicTrans is employed. NLLB provides broad multilingual coverage, while IndicTrans enhances accuracy for Indian and low-resource languages; together, they translate the normalized query into English, creating a unified semantic representation for retrieval. The translated query is processed through a hybrid search strategy that integrates exact substring matching, RapidFuzz-based fuzzy matching, and dense semantic retrieval using a MiniLM sentence transformer with cosine similarity scoring. A weighted ranking mechanism combines these signals to identify the most relevant plant record, which is then formatted into a structured response containing plant name, botanical name, medicinal uses, preparation methods, and active constituents. If no confident match is found, a fallback response is generated.

Input: User query (text or speech) in any supported global or Indian language.
Output: Structured medicinal plant information in a user-readable format (or fallback message if not found).

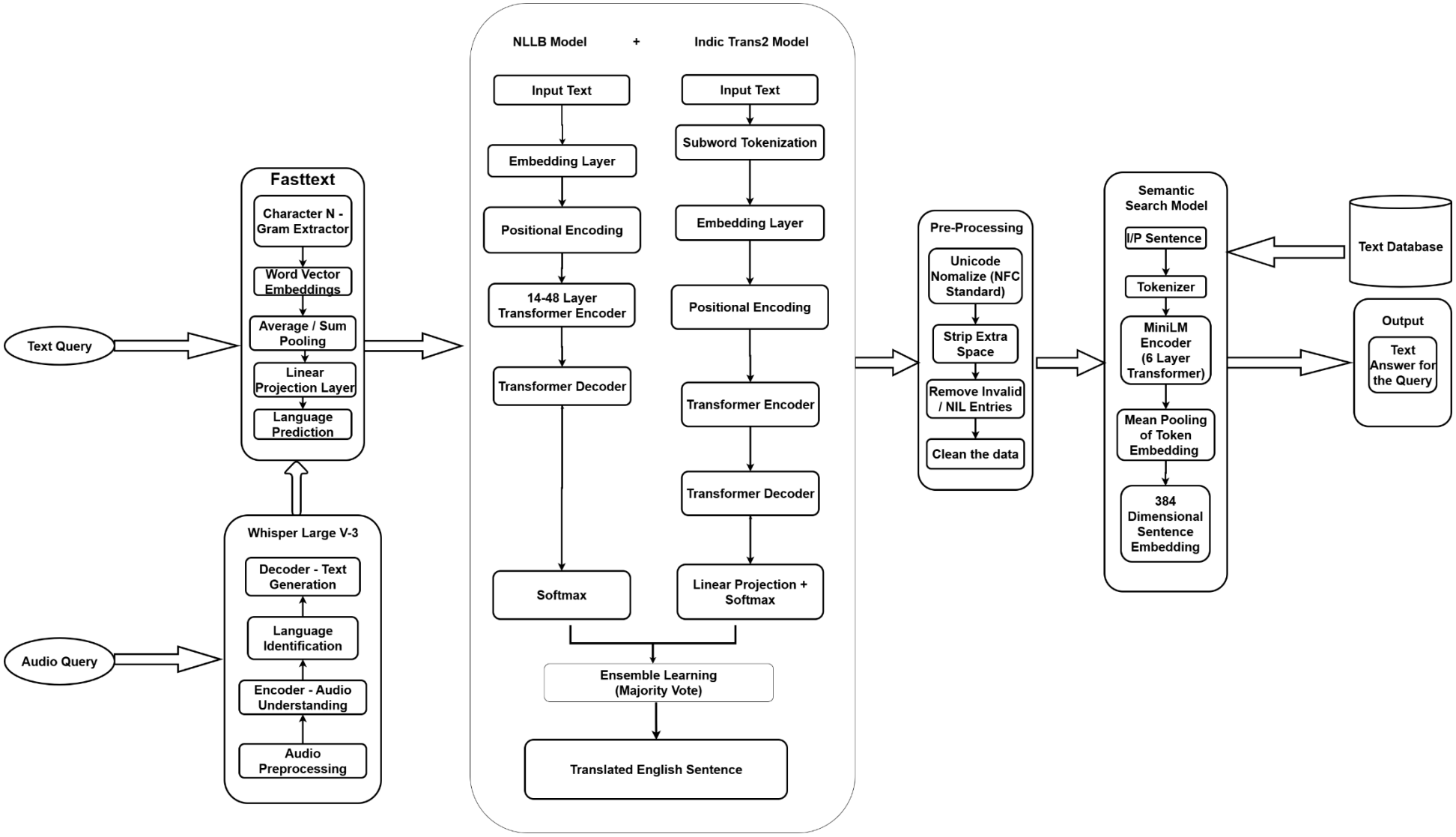

Figure 3.2 Module 1 – Query Processing and Semantic Retrieval

3.2.1 FastText Model

The FastText model is used as the initial component in the multilingual query processing pipeline to perform language detection and feature extraction from the user's query. FastText, developed by Facebook AI Research, is an efficient word representation and text classification model that works particularly well for multilingual environments. Unlike traditional word embedding models that treat each word as a single unit, FastText represents words using character-level n-grams, allowing the system to capture morphological patterns and handle rare or previously unseen words effectively. This property is especially useful when processing user queries in multiple Indian and international languages.

When a text query is provided by the user, it first enters the FastText module where the model begins by performing character n-gram extraction. In this step, each word in the query is broken down into smaller overlapping character sequences called n-grams. For instance, the word plant may be divided into segments such as <pl, pla, lan, ant, and nt>. These subword units help the model understand linguistic patterns within words, improving its ability to recognize variations, misspellings, and morphological forms across different languages.

After extracting these character n-grams, the system converts them into word vector embeddings. Each n-gram is mapped to a numerical vector representation in a high-dimensional space. These embeddings capture semantic and syntactic relationships between words, meaning that words with similar meanings tend to have vector representations that are close to each other in the embedding space. By representing words in this way, the model enables the system to better understand the semantic context of the query.

Once the embeddings are generated, the model performs an average or sum pooling operation. During this stage, all the individual n-gram vectors associated with a word or sentence are aggregated into a single fixed-length

vector. This pooling process ensures that queries of different lengths can be processed efficiently while maintaining consistent feature dimensions. The resulting pooled vector acts as a compact representation of the entire query.

The pooled vector is then passed through a linear projection layer, which transforms the extracted features into a format suitable for classification. This layer learns the optimal weights that help distinguish between different languages based on the input features. Through training on multilingual datasets, the projection layer becomes capable of identifying patterns that correspond to specific languages.

Finally, the processed features are fed into the language prediction layer, which classifies the input query into its corresponding language category. The predicted language label is a crucial output of the FastText module because it determines how the query will be handled in the next stages of the pipeline. Once the language is identified, the system routes the query to the hybrid translation module, where advanced neural machine translation models such as NLLB and IndicTrans2 are used to translate the query into English. This translation step enables the system to standardize queries from multiple languages into a common representation, allowing subsequent modules such as preprocessing, semantic embedding, and semantic search to operate effectively.

Algorithm 1 FastText Training Algorithm

```
Require: Training corpus D, learning rate α, embedding size d, epochs E
Ensure: Word embeddings W and classifier weights C
Initialize embeddings W and weights C randomly
Build vocabulary V from corpus D
for each word w ∈ V do
    Generate character n-grams of w
    Represent w as sum of its n-gram vectors
end for
for epoch = 1 to E do
    for each sentence S ∈ D do
        for each target word w_t in S do
            Extract context words around w_t
            Compute input vector from n-gram embeddings
            Predict output using Softmax classifier
            Compute loss between prediction and true label
            Update W and C using gradient descent
        end for
    end for
end for
return trained embeddings W, classifier weights C
```

Fig 3.2.1.1 Pseudo Code for Fasttext Model

3.2.2 Whisper Model

The Whisper model is used in the system to process audio-based user queries and convert them into text before further language processing takes place. Whisper is an advanced automatic speech recognition (ASR) model developed by OpenAI. It is designed to accurately transcribe speech from multiple languages and accents, making it highly suitable for multilingual applications. In this system, the Whisper Large V3 model is used because of its improved accuracy and robustness when dealing with noisy audio environments and diverse speech patterns.

When a user submits an audio query, the audio input first undergoes an audio preprocessing stage. In this stage, the raw audio signal is cleaned and prepared for model processing. The preprocessing process may include converting the audio into a standardized format, adjusting the sampling rate, and removing unnecessary background noise or silent segments. This ensures that the audio signal is optimized for accurate speech recognition.

After preprocessing, the audio signal is passed into the encoder component of the Whisper model. The encoder is responsible for analyzing the acoustic features of the speech signal and converting them into meaningful numerical representations. It extracts important characteristics such as phonetic patterns, speech rhythms, and acoustic structures from the audio waveform. These extracted features are then transformed into encoded representations that capture the semantic information contained in the spoken query.

Following the encoding stage, the model performs language identification. Since the system supports multiple languages, this step determines the language spoken in the audio query. Accurate language detection ensures that the system can properly process and translate the query in later stages of the pipeline. The Whisper model is capable of identifying a wide range of languages, making it highly effective for multilingual speech-based interactions.

Once the language has been identified, the encoded speech representation is passed to the decoder component, which performs text generation. The decoder converts the encoded audio features into a textual transcription of the spoken query. This transcription represents the exact spoken content in written form while maintaining the linguistic structure of the original speech.

The resulting transcribed text is then forwarded to the FastText language detection module. At this stage, FastText further verifies the language and extracts textual features required for subsequent translation. By converting audio queries into text first, the Whisper model enables the system to treat both text-based and audio-based inputs in a unified processing pipeline. This ensures that all queries, regardless of their input format, can be translated, semantically analyzed, and processed by the downstream modules such as translation models, preprocessing components, and the semantic search system.

Algorithm 2 Whisper Speech-to-Text Conversion

Require: Audio input A
Ensure: Transcribed text T
1: Load pretrained Whisper model
2: Convert audio into waveform representation
3: Extract speech features
4: $T \leftarrow model.transcribe(A)$
5: **return** transcription T

Figure 3.2.2.1 Pseudo Code for Whisper Model

3.2.3 Hybrid Translation Model

The Hybrid Translation Model is responsible for converting multilingual user queries into a standardized English representation so that the system can process the query effectively in later stages such as semantic search and information retrieval. Instead of relying on a single translation model, the system uses a hybrid architecture that combines the strengths of two advanced neural machine translation models: NLLB Model and IndicTrans2 Model. By

integrating these two models and applying ensemble learning, the system improves translation accuracy and robustness when handling multiple languages.

When a user query enters the hybrid translation module, the input text is simultaneously processed by both translation models. In the NLLB (No Language Left Behind) model, the input sentence first passes through an embedding layer, where each token in the sentence is converted into a numerical vector representation. These embeddings capture the semantic meaning of the words in the query. After embedding generation, the model applies positional encoding, which helps the system understand the order and position of words within the sentence. Since transformers do not inherently process sequences in order, positional encoding provides the necessary information about token positions.

Following this, the encoded representations are passed through a deep transformer encoder network, typically consisting of 14–48 transformer layers. Each layer performs attention-based computations that allow the model to understand contextual relationships between words in the sentence. The encoder captures both local and long-range dependencies within the text, enabling the system to interpret the meaning of the entire sentence rather than individual words.

Once the encoding process is complete, the encoded representations are forwarded to the transformer decoder. The decoder generates the translated output by predicting one token at a time while considering both the encoded input context and previously generated tokens. The final output of the decoder is passed through a softmax layer, which determines the most probable word for each position in the translated sentence.

Parallel to this process, the IndicTrans2 model processes the same input query. In this model, the first stage is subword tokenization, where the sentence is broken into smaller subword units to better handle rich morphological variations present in many Indic languages. These tokens are then passed through an embedding layer, where they are converted into vector

representations. Similar to the NLLB model, positional encoding is applied to maintain the sequential structure of the input text.

The embedded tokens are then processed by the transformer encoder, which analyzes the contextual relationships between words and extracts semantic features from the sentence. After encoding, the information is sent to the transformer decoder, which generates the translated English sentence. The decoder output is then processed through a linear projection layer followed by a softmax function to determine the most probable sequence of translated words.

Once both models produce their translation outputs, the system applies an ensemble learning strategy, specifically a majority voting mechanism. This approach compares the outputs from both models and selects the most reliable translation result. By combining the outputs of two powerful translation systems, the hybrid model reduces translation errors, improves consistency, and increases overall translation quality.

The final output of this module is a translated English sentence, which represents the standardized version of the user's original query. This translated query is then forwarded to the data preprocessing module, where further cleaning and normalization operations are performed before the query is processed by the semantic search model. By integrating multiple translation models, the hybrid translation module ensures accurate and reliable multilingual query understanding within the system.

Algorithm 3 Hybrid Translation using FastText, IndicTrans2, and NLLB

```
Require: Input text T
Ensure: English translated text E
(lang, confidence) ← FastText(T)
if lang = English then
    E ← T
else
    if lang belongs to Indic languages then
        T1 ← IndicTrans2(T)
    end if
    T2 ← NLLB(T)
    Compute scores:
    score1 ← Score(T1)
    score2 ← Score(T2)
    if score1 > score2 then
        E ← T1
    else
        E ← T2
    end if
end if
return final translation E
```

Figure 3.2.3.1 Pseudo Code for Hybrid Translation Model

3.2.4 Preprocessing Module

The Preprocessing module plays an important role in preparing the translated query for semantic analysis and information retrieval. After the Hybrid Translation Model converts the user's multilingual query into an English sentence, the text may still contain irregular characters, unwanted spaces, or invalid entries. The preprocessing stage ensures that the text is clean, standardized, and structured, allowing the downstream semantic search model to interpret the query accurately.

The first step in this module is Unicode normalization using the NFC (Normalization Form C) standard, which is defined by the Unicode Consortium. Unicode normalization ensures that characters that may appear in multiple encoded forms are converted into a consistent representation. This step is particularly important when dealing with multilingual text because certain characters in different languages may be represented using multiple Unicode combinations. By normalizing the text into a standard format, the system avoids inconsistencies during further processing.

Following normalization, the system performs extra space removal. During translation or user input, additional spaces may appear between words or at the beginning and end of the sentence. These unnecessary spaces can affect tokenization and semantic analysis. Therefore, the preprocessing module removes redundant spaces and ensures proper spacing between words, improving the readability and structural consistency of the text.

The next stage involves removing invalid or null entries. Sometimes the translated query may contain incomplete words, null characters, or irrelevant symbols that do not contribute to the meaning of the sentence. Such elements are filtered out to prevent them from affecting the performance of the semantic search model. This step ensures that only meaningful textual data is retained for further processing.

After eliminating invalid entries, the system proceeds with data cleaning. In this step, unwanted punctuation marks, special characters, and formatting inconsistencies are removed or standardized. The objective of this stage is to transform the query into a structured and machine-readable format while preserving the original meaning of the sentence. Clean data significantly improves the performance of machine learning models because it reduces noise and ambiguity within the input.

Once all preprocessing steps are completed, the result is a clean and normalized English query that accurately represents the user's original input. This processed text is then passed to the semantic search module, where it is converted into vector embeddings and compared with entries in the text database to retrieve the most relevant answer. By ensuring consistency and clarity in the input data, the preprocessing module plays a critical role in improving the overall accuracy and efficiency of the system.

3.2.5 Semantic Search Model

The Semantic Search Model is responsible for identifying the most relevant information from the knowledge base based on the meaning of the user's query rather than simple keyword matching. Traditional search systems rely on exact keyword matches, which often fail to capture the true intent of a user's query. In contrast, semantic search focuses on understanding the context and meaning of the query to retrieve more accurate and meaningful results. In this system, the semantic search mechanism is implemented using Sentence Transformers, particularly the all-MiniLM-L6-v2 model, which is widely used for generating high-quality sentence embeddings.

When the preprocessed English query enters the semantic search module, the first step is tokenization and embedding generation. The Sentence Transformer converts the entire query into a dense vector representation, also known as a sentence embedding. This embedding captures the semantic meaning of the sentence by representing it as a numerical vector in a high-dimensional

space. The model has been trained on large-scale datasets to understand relationships between words, phrases, and sentences, allowing it to represent semantically similar sentences with closely related vector representations.

Simultaneously, the system maintains a vectorized knowledge database that contains embeddings of plant-related information stored in the dataset. Each entry in the database, such as plant descriptions, medicinal uses, and characteristics, has already been converted into vector embeddings using the same Sentence Transformer model. By using the same embedding model for both queries and stored data, the system ensures that they exist in the same semantic vector space, enabling effective comparison.

Once the query embedding is generated, the system performs a similarity comparison between the query vector and the embeddings stored in the knowledge database. This comparison is typically performed using cosine similarity, which measures the similarity between two vectors based on the angle between them. Cosine similarity values range from -1 to 1, where values closer to 1 indicate a higher degree of similarity between the query and the stored data.

During this stage, the system calculates the similarity score between the query embedding and each stored embedding in the database. The entries with the highest similarity scores are considered the most relevant results for the user's query. These top-ranked results are then retrieved and passed to the response generation module.

The retrieved result contains structured plant information such as plant name, medicinal properties, uses, and other botanical details. This information is formatted and prepared for the final response stage. By using semantic embeddings instead of keyword matching, the system is capable of understanding queries that use different wording but express the same meaning.

Finally, the retrieved result is forwarded to the response generation module, where the answer is structured and prepared for presentation to the user. The output may also be translated back into the user's preferred language if required. Through this approach, the semantic search model ensures that users

receive accurate, context-aware, and meaningful responses, significantly improving the overall effectiveness of the system.

Algorithm 4 Semantic Retrieval using MiniLM-L6-v2

Require: User query Q, plant dataset D
Ensure: Best matching plant record R

```
Load MiniLM-L6-v2 embedding model
E_q ← Encode(Q)
for each plant description d ∈ D do
    E_d ← Encode(d)
    S_d ← CosineSimilarity(E_q, E_d)
end for
R ← argmax(S_d)
return best matching plant R
```

Figure 3.2.5.1 Pseudo Code for Semantic Search Model

```python
# ====================================================
# ENSEMBLE TRANSLATION (→ ENGLISH)
# ====================================================
def translate_with_scores(self, text: str) -> dict:
    if not text or not text.strip():
        return self._empty_result()

    labels, probs = self.lid_model.predict(text.replace("\n", " "), k=1)
    lang = labels[0].replace("__label__", "")
    lang_conf = float(probs[0])

    if lang == "en":
        return {
            "t1": text,
            "t2": text,
            "score1": 1.0,
            "score2": 1.0,
            "final": text
        }

    # IndicTrans2
    t1 = ""
    if lang in self.indic_map:
        t1 = self._translate_indic(text, self.indic_map[lang])

    # NLLB
    t2 = self._translate_nllb(
        text, self.nllb_map.get(lang, "eng_Latn")
    )

    score1 = self._score_translation(t1, lang_conf)
    score2 = self._score_translation(t2, lang_conf)

    final = t1 if score1 >= score2 else t2

    if self.use_cuda:
        torch.cuda.empty_cache()

    return {
        "t1": t1,
        "t2": t2,
        "score1": round(score1, 3),
        "score2": round(score2, 3),
        "final": final
    }

# ====================================================
```

The hybrid_translator.py is a Python-based module designed to implement a robust and intelligent multilingual translation pipeline using a hybrid ensemble approach. The core objective of this module is to translate user input from multiple Indian and global languages into English with improved accuracy by combining multiple state-of-the-art models. The system dynamically detects

the input language using FastText language identification (lid.176.bin) and routes the text through specialized translation models based on the detected language. For Indic languages, the module leverages the AI4Bharat IndicTrans2 model (ai4bharat/indictrans2-indic-en-1B), which is optimized for high-quality Indic-to-English translation. For other supported global languages, it utilizes Meta's NLLB-200 distilled model (facebook/nllb-200-distilled-600M) to ensure broad multilingual coverage. The class automatically detects GPU availability and intelligently configures device allocation (CUDA or CPU), optimizing performance and memory usage with half-precision (float16) inference when GPU is available. The translate_with_scores function performs ensemble translation by generating outputs from both models (where applicable), computing confidence-based scores, and selecting the best final translation. If the detected language is already English, the system bypasses translation to preserve efficiency. Additionally, the architecture includes structured preprocessing, controlled inference handling, and optional GPU memory cleanup to maintain stability during deployment. Overall, hybrid_translator.py provides a scalable, production-ready hybrid translation framework that enhances translation reliability by combining language identification, model specialization, confidence scoring, and dynamic model selection into a unified pipeline.

3.3 IMAGE BASED SEMANTIC FEATURE EXTRACTION AND SEARCH

The Image based semantic feature extraction and search system implements an integrated vision–language architecture that combines deep convolutional neural networks with transformer-based semantic retrieval to perform accurate plant identification and knowledge association. The process begins with a plant image captured from a camera or image database, which undergoes preprocessing steps including resizing, color normalization, pixel scaling, and tensor conversion to ensure robustness against variations in lighting, orientation, and background noise. The processed image is passed through a

hybrid deep learning backbone composed of EfficientNetv2B0 and ResNet50V2, where EfficientNetv2B0 extracts fine-grained morphological features and ResNet50V2 captures deep residual representations for discriminative learning. Feature maps from both networks are pooled, fused into a unified embedding, and passed through a classification head to predict the plant species. To enrich the prediction with meaningful information, the visual embedding is projected into a shared semantic space aligned with text embeddings generated using a MiniLM-based transformer encoder. Cosine similarity is used to retrieve the most relevant botanical and medicinal descriptions from the database. The final output presents both the predicted plant species and structured semantic information, enabling accurate, interpretable, and knowledge-driven plant identification.

- **Input:** Plant image (from camera or image dataset).
- **Output:** Predicted plant species name along with structured botanical and medicinal information.

•

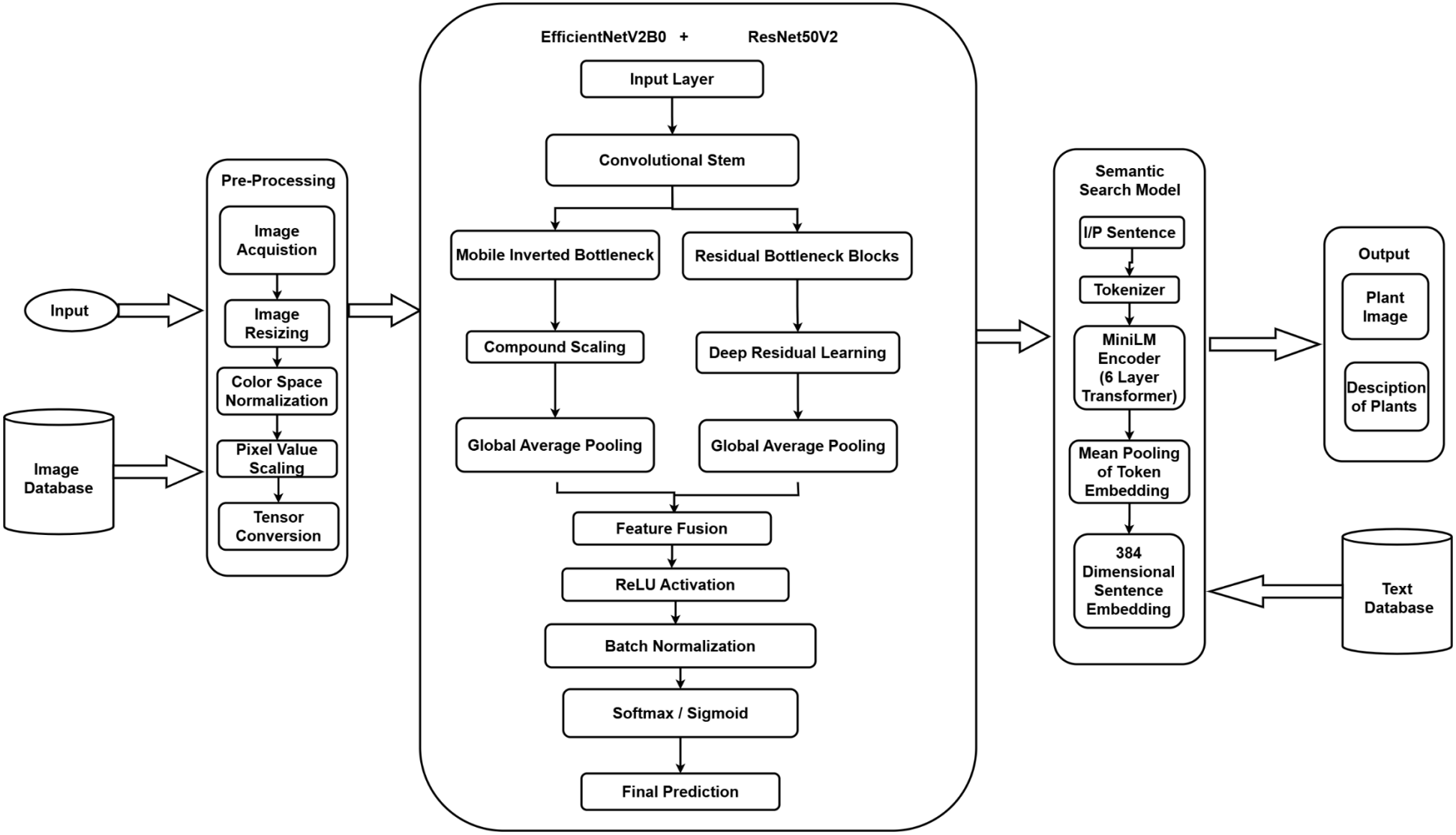

Figure 3.3 Module 2 – Hybrid Vision Model

3.3.1 Image Preprocessing

The Image Preprocessing module is the first stage in the plant image recognition pipeline. Its main objective is to prepare raw input images in a structured format suitable for deep learning models. Images collected from users or stored in the image database may vary in resolution, lighting conditions, and color representation. Therefore, preprocessing is necessary to standardize these images before they are passed to the convolutional neural network models for feature extraction and classification.

The first step in this module is image acquisition. In this stage, the system receives plant images either from user uploads through the application interface or from the internal image dataset used for training and testing. These images may originate from different devices such as mobile cameras or online datasets, which means they may have varying dimensions, orientations, and quality levels. Proper acquisition ensures that the image data is successfully loaded into the system for further processing.

After acquisition, the system performs image resizing. Deep learning models require images to have a fixed input dimension so that the convolutional layers can process them efficiently. During this step, all input images are resized to a consistent resolution, typically 224 × 224 pixels, which is a common input size for convolutional neural networks such as EfficientNetV2B0 and ResNet50V2. Resizing ensures uniformity across all images and reduces computational complexity while preserving essential visual features of the plant.

The next stage is color space normalization. Images are typically represented in the RGB color format, but variations in lighting conditions or camera settings can lead to inconsistencies in color distribution. In this step, the color values of the image are normalized to ensure that the model focuses on meaningful visual patterns rather than variations caused by environmental conditions. This normalization improves the stability and reliability of the

feature extraction process.

Following color normalization, the system performs pixel value scaling. Raw pixel values in an image usually range from 0 to 255. To make the data suitable for neural network processing, these values are scaled to a smaller range, typically 0 to 1. Scaling helps stabilize the training process and improves convergence during model learning by preventing extremely large input values from affecting the gradient calculations.

Finally, the preprocessed image is converted into a tensor representation. Deep learning frameworks such as PyTorch and TensorFlow process data in the form of tensors rather than standard image arrays. Tensor conversion restructures the image data into a multidimensional numerical format that the neural network can efficiently process during forward propagation.

After completing all preprocessing steps, the image becomes a clean, normalized, and standardized input ready for the feature extraction stage. This processed image is then forwarded to the hybrid deep learning architecture that combines EfficientNetV2B0 and ResNet50V2 models for plant identification and classification. Proper preprocessing ensures that the neural network receives high-quality input data, which significantly improves the accuracy and reliability of plant recognition results.

Algorithm 5 Image Preprocessing Pipeline

Require: Input leaf image I
Ensure: Normalized tensor image I_{proc}

```
Resize image to fixed resolution (e.g., 224 × 224)
Apply data augmentation:
    Random rotation
    Horizontal flip
    Color normalization
    Random cropping
Convert image into tensor format
Normalize pixel values using mean and standard deviation
I_proc ← Preprocessed image tensor
return I_proc
```

Figure 3.3.1.1 Pseudo Code for Image Preprocessing

3.3.2 Hybrid Vision Model

The Hybrid Vision Model is the core component responsible for identifying plant species from the preprocessed input image. In this system, a hybrid deep learning architecture is used by combining two powerful convolutional neural network models: EfficientNetV2B0 and ResNet50V2. The purpose of integrating these two models is to leverage the strengths of both architectures to improve feature extraction and classification accuracy for plant identification tasks.

The process begins with the input layer, where the preprocessed image tensor obtained from the preprocessing module is fed into the hybrid model. This input image typically has a fixed dimension (for example, 224 × 224 × 3) and serves as the starting point for the convolutional feature extraction process.

Next, the image passes through the convolutional stem, which performs the initial feature extraction. This stage applies convolutional filters to detect fundamental visual patterns such as edges, textures, and color variations. The convolutional stem reduces spatial dimensions while increasing the depth of the feature maps, enabling the model to capture basic structural information from the plant image.

After the convolutional stem, the architecture splits into two parallel pathways corresponding to the two backbone models. In the EfficientNetV2B0 branch, the feature maps are processed through mobile inverted bottleneck (MBConv) blocks. These blocks are designed to efficiently capture complex visual patterns while maintaining computational efficiency. EfficientNet uses a technique known as compound scaling, which balances network depth, width, and resolution to achieve better performance with fewer parameters. Through these layers, the model extracts detailed features such as leaf patterns, shapes, and texture characteristics that are important for plant recognition.

In parallel, the ResNet50V2 branch processes the same feature maps through residual bottleneck blocks. ResNet introduces skip connections that allow the network to learn deep representations without suffering from the vanishing gradient problem. These residual connections enable the model to capture deeper hierarchical features, such as complex plant structures and vein patterns within leaves.

Following feature extraction in both branches, the outputs are passed through global average pooling layers. This operation reduces each feature map into a single representative value, effectively summarizing the most important visual features detected by the network while significantly reducing the number of parameters.

The pooled features from both networks are then combined using a feature fusion layer. This fusion process integrates the complementary strengths of EfficientNet and ResNet by merging their extracted features into a unified representation. The fused feature vector contains both efficient spatial features and deep residual features, providing a richer description of the plant image.

After fusion, the combined features are passed through a ReLU activation layer, which introduces non-linearity into the model. This allows the neural network to learn complex patterns and relationships within the data. The activated features then undergo batch normalization, which stabilizes the training process and improves convergence by normalizing the distribution of feature values.

Finally, the processed features are passed to a classification layer that uses either a softmax or sigmoid activation function depending on the classification setup. The softmax function is typically used for multi-class classification, where the model predicts the probability of the input image belonging to a specific plant species. The output of this layer produces the final prediction, identifying the most likely plant class based on the visual features extracted from the image.

The predicted plant label is then forwarded to the semantic search module, which retrieves detailed textual information about the identified plant from the knowledge database. By combining EfficientNetV2B0 and ResNet50V2 within a hybrid architecture, the system achieves higher accuracy and robustness in plant recognition compared to using a single model alone.

Algorithm 6 Hybrid Vision Model using EfficientNetV2-B0 and ResNet50V2

Require: Preprocessed image I_{proc}
Ensure: Predicted plant class label C

```
Load pretrained EfficientNetV2-B0 backbone
Remove classification head
Load pretrained ResNet50V2 backbone
Remove classification head
Extract feature vectors:
F_eff ← EfficientNet(I_proc)
F_res ← ResNet(I_proc)
Fuse feature representations:
F_fusion ← Concatenate(F_eff, F_res)
Pass fused vector through fully connected layer
Apply Softmax classifier:
C ← argmax(Softmax(F_fusion))
return predicted plant label C
```

Figure 3.3.2.1 Pseudo Code for Hybrid Vision Model

```python
def __init__(self, num_classes=1081, freeze_backbones=True):
    super().__init__()

    # -------------------------------
    # BACKBONE 1: EfficientNetV2-B0
    # -------------------------------
    self.effnet = timm.create_model(
        "tf_efficientnetv2_b0",
        pretrained=True,
        num_classes=0,        # remove classifier
        global_pool="avg"     # ensure flat feature vector
    )

    # -------------------------------
    # BACKBONE 2: ResNet50V2
    # -------------------------------
    self.resnet = timm.create_model(
        "resnetv2_50",
        pretrained=True,
        num_classes=0,
        global_pool="avg"
    )

    eff_dim = self.effnet.num_features
    res_dim = self.resnet.num_features
    fused_dim = eff_dim + res_dim
```

This hybrid model combines EfficientNetV2-B0 and ResNet50 as dual backbone feature extractors to enhance representation learning. Both pretrained networks are initialized without their classification heads, enabling extraction of high-level feature vectors through global average pooling. The features from EfficientNet and ResNet are then fused (concatenated) to form a richer combined representation. By merging the lightweight efficiency of EfficientNet with the deep residual learning strength of ResNet, the model improves robustness and classification performance. This architecture leverages complementary feature learning for more accurate and generalized predictions.

3.4 GENERATIVE MODEL

This Generative Model implements a structured NLP pipeline to interpret user queries, generate accurate plant-related responses, and deliver them in the user's preferred language with semantic consistency. The process begins with intent prediction, where keyword extraction identifies important plant-related terms and maps them to predefined intent categories such as plant identification, medicinal uses, or general botanical information. Based on the detected intent, relevant plant facts are retrieved from a curated knowledge base and inserted into predefined prompt templates during the prompt construction stage, ensuring factual grounding and standardized response formatting. The structured prompt is then processed by a Flan-T5-based text generation model, which performs constrained generation to produce coherent, context-aware, and fact-aligned responses while minimizing hallucination. Once the English response is generated, it is passed to a hybrid translation module combining NLLB and Indic Translation 2. NLLB ensures broad multilingual coverage, while Indic Translation 2 enhances accuracy for Indian and low-resource languages. The hybrid mechanism translates the response back into the exact language originally typed by the user, maintaining semantic integrity and linguistic fluency. The final output is a clear, structured, and multilingual plant-related response aligned precisely with the user's intent.

Input: User query (text) in any supported language.

Output: Accurate, structured plant-related response translated into the user's originally typed language.

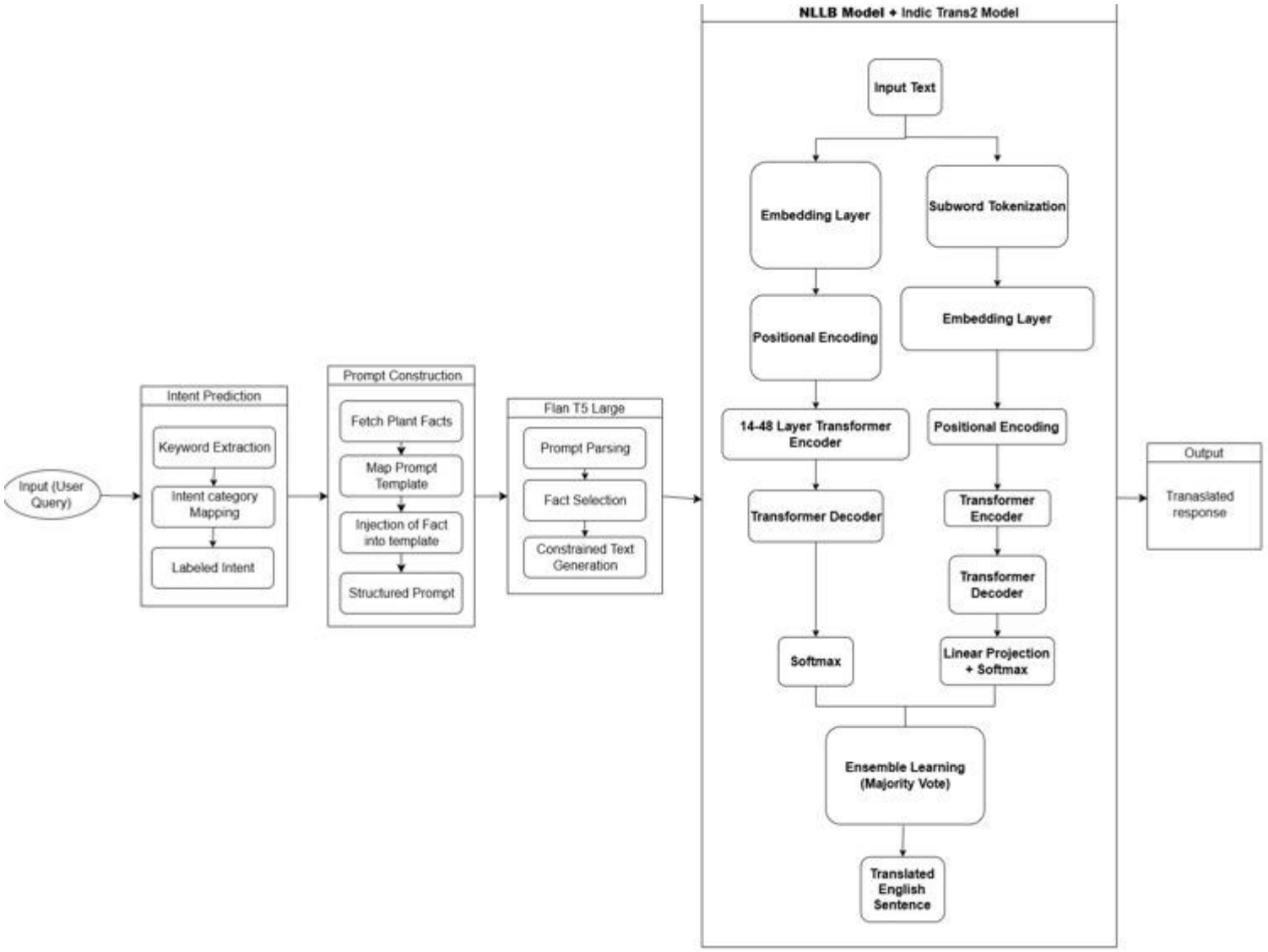

Figure 3.4 Module 3 – Generative Model

3.4.1 Intent Prediction

The Intent Prediction module is responsible for understanding the purpose of the user's query before generating a response. When a user submits a query through the system, the query may contain different types of requests such as asking about medicinal uses of a plant, plant identification, benefits, preparation methods, or general botanical information. Therefore, it is necessary to determine the user's intent so that the system can retrieve and generate the most relevant information.

The process begins with keyword extraction from the user query. In this step, important terms and phrases related to plants, diseases, medicinal properties, or usage instructions are identified from the input text. The system filters out unnecessary words and focuses on meaningful keywords that indicate the user's request. For example, words such as benefits, medicinal uses, how to use, or identify plant help the system understand what type of information the user is seeking.

After extracting the keywords, the system performs intent category mapping. During this stage, the extracted keywords are mapped to predefined intent categories within the system. These categories may include plant identification, medicinal usage, health benefits, preparation methods, or general plant descriptions. Mapping the query to a specific category allows the system to organize user requests into structured groups and simplifies the response generation process.

Once the mapping is completed, the system produces a labeled intent. This labeled intent represents the final classification of the user's query. For instance, if the user asks "What are the medicinal uses of Tulsi?", the system labels the intent as Medicinal Uses Inquiry. Similarly, a query like "How to prepare neem leaves for skin treatment?" may be labeled as Preparation Method Inquiry.

Identifying the correct intent is important because it determines how the system will construct the response in the subsequent stages. The labeled intent is forwarded to the prompt construction module, where relevant plant facts and structured prompts are generated based on the identified intent. By accurately predicting the user's intent, the system ensures that the generated responses are context-aware, relevant, and aligned with the user's information needs.

Algorithm 7 Intent Detection Module

Require: User query Q
Ensure: Detected intent category I

```
Preprocess query text
Convert query into embedding representation
Compare query embedding with predefined intent tem-
plates:
    Plant identification
    Medicinal usage request
    Translation request
    General plant information request
Compute similarity scores
Select highest matching intent:
I ← argmax(similarity)
return detected intent I
```

Figure 3.4.1.1 Pseudo Code for Intent Prediction

3.4.2 Prompt Construction

The Prompt Construction module is responsible for preparing a structured prompt that will be used by the language generation model to produce an accurate and context-aware response. After the Intent Prediction module identifies the user's intent, the system needs to gather relevant plant information and organize it in a structured format. This structured prompt helps the downstream language model understand the context of the query and generate a precise answer.

The first step in this process is fetching plant facts from the internal knowledge base. Once the intent is identified, the system retrieves relevant information related to the plant from the database. These facts may include the

plant's scientific name, medicinal properties, traditional uses, preparation methods, and health benefits. The retrieved facts serve as the factual foundation that will guide the response generation process.

After retrieving the necessary plant information, the system performs prompt template mapping. In this step, the system selects an appropriate prompt template based on the predicted intent category. Each intent category has a predefined template structure designed to guide the language model in generating the correct type of response. For example, if the intent relates to medicinal benefits, the template will emphasize therapeutic uses and health-related information. If the query is related to plant identification, the template will focus on descriptive botanical features.

Once the appropriate template is selected, the system performs fact injection into the template. During this stage, the retrieved plant facts are inserted into specific sections of the prompt template. This ensures that the prompt contains structured and relevant information about the plant while maintaining a consistent format. The injected facts act as constraints that help the language model generate responses grounded in verified data rather than producing vague or unrelated information.

Finally, the system generates a structured prompt, which combines the user's query, the predicted intent, and the retrieved plant facts in a well-organized format. This structured prompt acts as the input for the response generation model. By organizing the information in this manner, the system ensures that the language model focuses only on relevant information when generating the response.

The constructed prompt is then forwarded to the response generation module, where a large language model processes the prompt and produces the final textual response. Through this structured prompt construction approach, the system improves the accuracy, relevance, and reliability of the generated answers while maintaining consistency across different types of user queries.

Algorithm 8 Prompt Construction for FLAN-T5

Require: Plant name P
Ensure: Structured prompt $Prompt$

```
Initialize instruction template
Add task description:
    Generate academic medicinal plant information
Add constraints:
    Include therapeutic uses
    Include supported health conditions
    Include nutritional value
    Avoid taxonomy and history
    Avoid unsupported claims
Limit response length to 4–6 sentences
Construct final prompt:
Prompt ← Template(P)
return generated prompt
```

Figure 3.4.2.1 Pseudo Code for Prompt Construction

3.4.3 FLAN-T5 Model

The FLAN-T5 model is used as the main response generation component in the system. After the prompt construction module prepares a structured prompt containing the user query, predicted intent, and relevant plant facts, this prompt is passed to the FLAN-T5 model to generate a meaningful and informative response. The model used in this system is FLAN-T5 Large developed by Google Research. FLAN-T5 is an instruction-tuned version of the T5 architecture that is specifically designed to follow structured prompts and produce high-quality natural language responses.

The process begins with prompt parsing, where the FLAN-T5 model analyzes the structured prompt generated in the previous stage. The prompt typically contains the user's question, contextual plant information retrieved from the database, and the predicted intent label. During parsing, the model interprets the prompt structure and identifies key components such as the query, the relevant facts, and the expected type of response. This helps the model understand what type of answer needs to be generated.

After understanding the prompt, the system performs fact selection. Although the prompt may contain multiple pieces of plant information, not all facts may be necessary for answering the query. In this stage, the model identifies the most relevant facts from the provided context. These selected facts guide the response generation process and ensure that the output remains accurate and

focused on the user's question.

Once the relevant information has been identified, the model performs constrained text generation. FLAN-T5 generates the response by using the selected facts while maintaining the context and structure defined in the prompt. The model follows the transformer-based encoder–decoder architecture, where the encoder processes the input prompt and converts it into contextual embeddings, and the decoder generates the output text token by token. During this generation process, the model ensures that the output remains consistent with the given plant facts and the predicted user intent.

The generated output is a natural language response that explains the plant information requested by the user. This response may include details such as medicinal properties, usage methods, benefits, or botanical descriptions depending on the query. Because the model is instruction-tuned, it is capable of producing responses that are structured, coherent, and contextually accurate.

After the response is generated, it is forwarded to the translation module, where the system translates the generated English response into the user's preferred language using the hybrid translation system. This ensures that users can receive answers in their native language, improving accessibility and usability of the system. By integrating the FLAN-T5 model into the pipeline, the system is able to produce intelligent, fact-guided, and context-aware responses for a wide variety of plant-related queries.

Algorithm 9 Medicinal Plant Information Generation using FLAN-T5

Require: Prompt $Prompt$
Ensure: Generated plant description $Output$

```
Load pretrained FLAN-T5 model
Load tokenizer
Convert prompt into token representation:
Tokens ← Tokenizer(Prompt)
Generate response using sequence-to-sequence inference:
Output ← Model.generate(Tokens)
Decode generated tokens into readable text
Post-process output for formatting consistency
return generated plant description
```

Figure 3.4.3.1 Pseudo Code of Flan T-5 Model

```python
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# ---------------------------------------------------
# DEVICE & MODEL
# ---------------------------------------------------
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
MODEL_NAME = "google/flan-t5-large"


# ---------------------------------------------------
# LOAD FLAN MODEL (ONCE)
# ---------------------------------------------------
@torch.inference_mode()
def load_flan():
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
    model.to(DEVICE)
    model.eval()
    return tokenizer, model


# ---------------------------------------------------
# GENERATE PLANT INFORMATION (STABLE + SAFE)
# ---------------------------------------------------
def generate_plant_info(tokenizer, model, plant_name: str):
    """
    Generates medically relevant, non-hallucinated plant information.
    Designed for academic / FYP usage.
    """

    prompt = f"""
Write a concise academic paragraph about the plant "{plant_name}".

Focus on:
- traditional medicinal uses
- health conditions it is commonly used to support
- nutritional or therapeutic value

Rules:
- Do NOT mention geography, history, or taxonomy.
- Do NOT invent scientific studies or timelines.
- Do NOT claim guaranteed cures.
- Use safe wording like "traditionally used", "may help", "used to support".

Write exactly 4 to 6 complete sentences.
"""
```

This module code integrates the FLAN-T5 Large language model to generate controlled, academically structured plant information. It uses lazy loading and GPU optimization (FP16 when available) to ensure efficient inference. The system includes preprocessing functions for text condensation and keyword extraction to improve response quality. Additionally, a structured prompt design enforces safe, non-hallucinated, medically relevant content generation suitable for everything. The architecture ensures stable and consistent outputs by operating in evaluation and inference modes. Overall, the module provides a reliable and scalable NLP component for intelligent plant knowledge generation within the system.

CHAPTER 4
RESULTS AND DISCUSSION

4.1. DATASET DESCRIPTION

The dataset used in this study was provided by the National Institute of Siddha (NIS), Chennai, an autonomous institution under the Ministry of AYUSH, Government of India. NIS maintains authoritative and systematically curated documentation of medicinal plants referenced in Siddha pharmacognosy, classical medical literature, and traditional therapeutic formulations. The dataset supplied by the institute consisted of approximately 200 medicinal plant records (rows), with each record described using around 40–45 structured attributes (columns) capturing scientific, linguistic, cultural, and pharmacological information essential for medicinal plant research and digital knowledge systems.

The spreadsheet included multiple fields such as the botanical name (Latin nomenclature), dataset or image reference links, Sanskrit names, regional language names (Tamil, Malayalam, Telugu, Kannada, and Hindi), and English common names. In addition, it contained detailed descriptive attributes including plant taxonomy, botanical and morphological descriptions, traditional therapeutic uses, recommended dosages, pharmacological actions, active constituents, and links to authenticated plant photographs. Given that Siddha medicine relies heavily on region-specific plant nomenclature, indigenous terminology, and classical references, this multilingual and multi-attribute dataset provided by NIS forms a highly reliable foundation for building semantic models, transliteration pipelines, and cross-lingual medicinal plant retrieval systems.

To enable efficient computational processing, a canonical data schema was designed to standardize and organize the dataset for downstream AI-based tasks. This schema ensures compatibility with relational databases, NoSQL systems, and embedding-based semantic search frameworks. Each plant entry is assigned a unique numeric identifier (plant_no) to ensure consistent referencing. The

botanical_name field serves as the authoritative global identifier, while english_name and multiple regional language fields (Tamil, Telugu, Malayalam, Kannada, and Hindi) support multilingual querying. Additional textual fields include concise botanical descriptions, documented medicinal usages, active phytochemical constituents, and classical Siddha literary references such as Tamil medicinal poems, along with their Tamil and English explanations. This structured schema enables a hierarchical and semantically rich representation of the NIS dataset, facilitating seamless integration with natural language models, semantic search engines, and multimodal plant information systems.

Since the dataset originated from manually curated records maintained by the National Institute of Siddha, a systematic cleaning and preprocessing pipeline was applied to ensure machine readability, consistency, and robustness. The preprocessing process began with data normalization, where leading and trailing whitespace was removed, inconsistent capitalization was corrected, and Unicode normalization (NFC/NFKC) was applied across all textual fields. This step is particularly important for Indian language scripts, which may include visually similar characters or diacritical variations that could otherwise be interpreted as distinct tokens by embedding models.

Missing values appeared in multiple formats such as “nil,” “NIL,” “None,” and empty cells, all of which were standardized to a uniform null representation. Duplicate plant entries—often present in manually compiled datasets—were identified and resolved using a combination of botanical name matching, fuzzy string similarity, and cross-lingual alias comparison across regional name fields. Removing duplicate entries improves embedding quality and prevents skewed similarity scores during semantic retrieval.

A critical preprocessing step involved constructing a combined all_text representation for each plant record. This field was created by concatenating the botanical name, English and regional names, botanical descriptions, traditional medicinal uses, active constituents, and explanatory notes. This aggregated textual representation provides rich semantic context, significantly enhancing the

effectiveness of embedding-based retrieval systems. Using this combined text, an offline embedding index was generated using pretrained sentence-transformer models, enabling efficient similarity search and ranking during query-time execution.

Because the dataset includes multiple non-Latin scripts such as Tamil, Malayalam, Telugu, and Kannada, pre-generated transliterations were created for all vernacular plant names. Transliteration tools such as indic-transliteration libraries and phonetic mapping strategies were employed to generate Romanized equivalents.

The cleaned and standardized dataset was exported into structured formats suitable for downstream integration, including CSV for lightweight storage and prototyping, and vector-compatible formats (e.g., Elasticsearch or FAISS-ready structures) for semantic search. These preprocessing steps collectively transform the 200-row, 40+-column NIS dataset into a robust, standardized, and semantically enriched resource, ready for use in multilingual plant identification systems, AI-driven knowledge retrieval pipelines, and interactive medicinal plant information platforms.

4.2 IMPLEMENTATION DETAILS

4.2.1 Folder structure

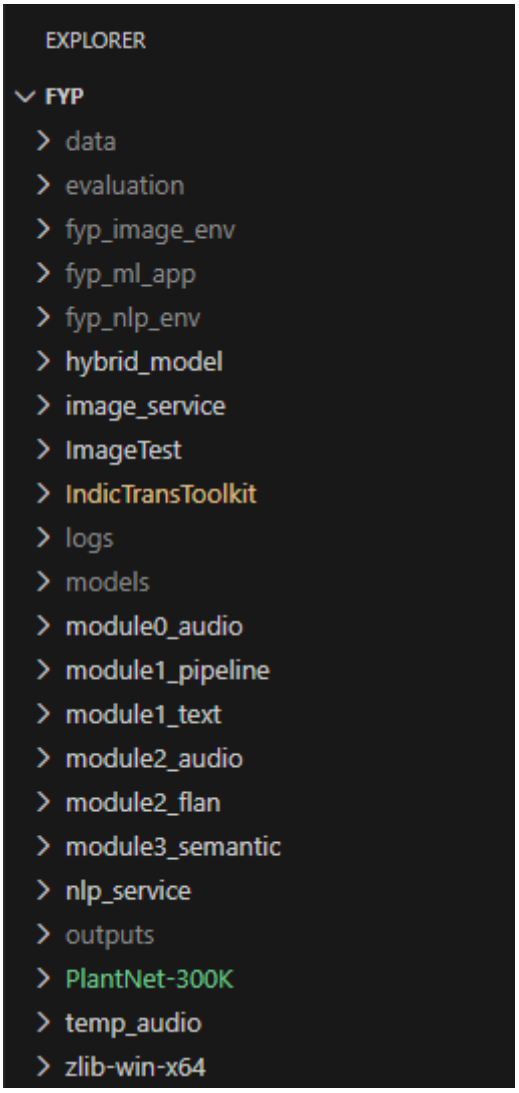

Figure 4.2.1 Folder Structure

Core Application Files

- app.py

  The main entry point of the Flask web application. It handles routing, file uploads, user interaction, and integration with backend modules.

- main_application.py

  The central logic orchestrator. This file integrates various modules — processing, summarization, enhancement, and visualization — to generate complete outputs from uploaded reports.

Templates for Web Interface

• app.py (Backend)

This Python file functions as the backend service responsible for loading and running the AI models used in the system. It handles tasks such as model initialization, image preprocessing, plant classification, translation processing, and response generation. The backend also exposes APIs or endpoints that allow the frontend interface to communicate with the machine learning models and retrieve results.

• React Frontend (User Interface)

The user interface of the system is developed using React, which provides an interactive and responsive web environment for users. The React application includes components such as text input fields, image upload options, audio input, language selection dropdowns, and action buttons that allow users to interact with the system. When a user submits input (text, image, or audio), the React frontend sends the request to the Python backend, which processes the data using the trained models and returns the results to be displayed on the interface.

4.2.2 Code Implementation

1. lang_detect.py

Implementation Details

- **Purpose**: It detects the language of the given user query and assigns label to it.
- **Libraries Used**: fasttext , pandas ,numpy

- **Core Methods**:
    - Detect_language(): It uses the fasttext model and then it'll identify the language type .

Results

- Successfully detected language of the given user query.
- Detected almost 90% of the given user queries correctly as per evaluation metrics.

2. Preprocess.py Implementation

Details

- **Purpose**: Removes unwanted characters(like ?,!,) and spaces .
- **Libraries Used**: unicodedata , re
- **Core Methods**:
    - clean_text(): Normalizes the user text, removes extra spaces, and returns the final completed text for the next process.

Result

- Gives clean text without any noise so that Semantic Search (Mini LM) model will able to detect the intent correctly .

3. Hybrid_translator.py

Implementation Details

- **Purpose**: Translates given user query language into English Language(Base Language)
- **Libraries Used**: transformers,AutoTokenizer,AutoModelForSeq2SeqLM
- **Core Methods**:
    - Translate_indic(): Translates only 22 Indian Languages to English
    - Translate_nllb(): Translates all global languages except Indian Language into English.
    - Score_translation(): Gives Score for both translation models and takes the best as per Semantic similarity.

Results

- Translated the given user query to English Language.
- Translated the query into English with > 90% accuracy.

4. Build_embeddings.py

Implementation Details

- **Purpose**: Converts the given text into text embeddings.
- **Libraries Used**: pandas,embed
- **Core Methods**:
    - Embed(): Converts text to embeddings .

Result:

- Sucessfully converted user text to contextual embeddings.

5. search.py

Implementation Details

- **Purpose**: Searches in the dataset as per predicted intent .
- **Libraries Used**: sentence_transformers,util,torch
- **Core Methods**:
    - Semantic _search(): Matches query embeddings with dataset embeddings and gives top 1 matching result .

Result

- Sucessfully extracted resultant answer with > 70% accuracy.

6. Train_hybrid.py.

Implementation Details

- **Purpose**: Trains the model with given 1080 species with augmented images with hybrid model by combining 2 models of EfficientNetV2B0 and ResNet50V2 .
- **Libraries Used**: torch,neural network(nn)

Result

- Trained the model with accuracy of about >85%.

7. Flan_generator.py

Implementation Details

- **Purpose**: Given set of rules in such a way that it does not Hallucinate.
- **Libraries Used**: transformers,AutoTokenizer,AutoModelForSeq2SeqLM
  - **Core Methods**:
    - Generate_plant_info():Given set of rules to the model in a such a way it should be satisfied in order to generate content from the dataset only and moreover it should not hallucinate.

Results

- Generate the content from the dataset effectively without any hallucination.

4.3 Output:

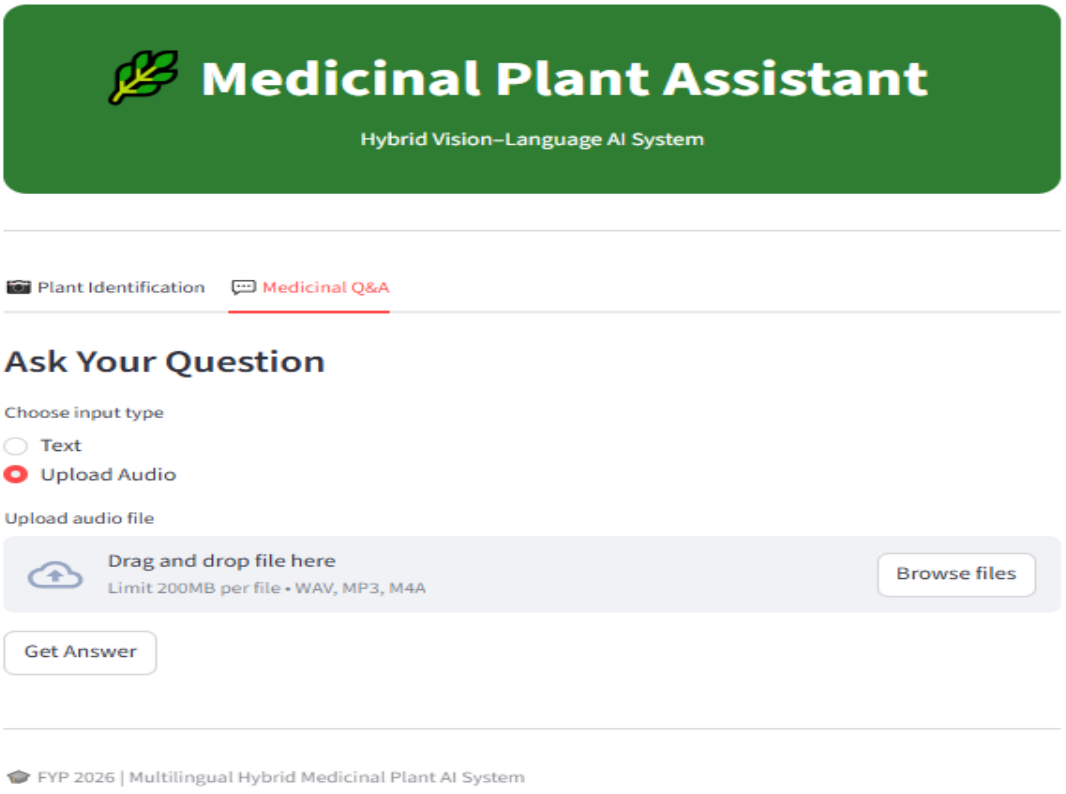

Fig 4.3.1 Home screen

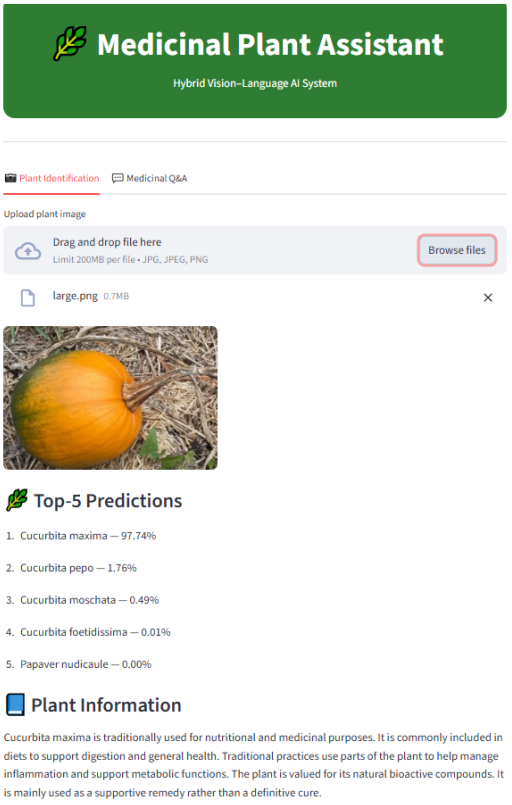

Fig 4.3.2 Output for image input

Fig 4.3.3 Output for text input

4.4. TEST CASES

Table 4.1 Test Cases

TEST CASE ID	OBJECTIVE	INPUT	EXPECTED OUTPUT
1.Testing Plant Information Generation Module	1.1: Generate plant information	Neem	4-6 sentence academic paragraph describing medicinal uses of the plant.
	1.2: Generate info with Uncommon plant	Ashwagandha	Structured paragraph mentioning therapeutic value safety.
	1.3: Handle unknown plant name	XYZ	Fallback message: “Information not available”
	1.4: Handle empty input	“”	Error : “Plant name required”
2. Testing Hybrid Translation Module	2.1: Translate Tamil to English	துளசி உடல்நலத்திற்கு உதவும்	Correct English translation about Tulsi.

	2.2: Translate Hindi to English	अश्वगंधा तनाव कम करने में मदद करता है	Proper English Translation
	2.3: Handle English Input	Tulsi improves immunity	Same Text returned (No Translation)
	2.4: Handle empty input	“ ”	Empty result or Structured Error Response/
3.Testing Language Detection	3.1: Detect Tamil Language	Tamil sentence	Language Detected : “ta” .
	3.2: Detect Hindi Language	Hindi Sentence	Language Detected : “hi”
	3.3: Detect English Language	English Sentence	Language Detected : “en”
	3.4: Handles noisy text query	###123@Tulsi	Error

4. Testing Image Classification	4.1: Classify Tulsi Imgae	Valid Tulsi image	Predicted class : Tulsi
	4.2: Classify Neem Image	Neem leaf image	Predicted class : Neem
	4.3: Handles unclear image	Blurry plant image	Lower confidence prediction
	4.4: Handle non plant image	Random object image	Incorrect plant class or rejection message

Table 4.1 presents the test cases designed to evaluate the functionality and reliability of the medicinal plant assistant system across its major modules. The plant information generation module is tested for its ability to produce structured academic descriptions for known plants such as Neem and Ashwagandha, while also handling unknown or empty inputs with appropriate fallback or error messages. The hybrid translation module verifies accurate translation from Tamil and Hindi to English, ensures English input remains unchanged, and checks responses for empty inputs. The language detection module evaluates whether the system correctly identifies Tamil, Hindi, and English text and handles noisy or invalid queries effectively. Finally, the image classification module assesses the system's capability to correctly recognize plant images like Tulsi and Neem, provide lower confidence predictions for unclear images, and respond appropriately to non-plant images. Overall, these test cases ensure the system's accuracy, robustness, and proper error handling across all modules.

4.5. PERFORMANCE METRICS

Following are the metrics used to evaluate the results of our model:

1) PRECISION

Measures the proportion of correctly predicted positive observations to the total classification , translation and similarity based metrics.

$$Precision = \frac{TP}{TP + FP}$$

Where:

TP = True Positives

FP = False Positives

--- (1)

2) BLEU SCORE

Evaluates the fluency and adequacy of generated summaries by comparing them to reference translations.

$$BLEU = BP \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Where:

- BP = **Brevity Penalty**, used to penalize overly short summaries.
- w_n = **Weight for each n-gram precision score** (commonly equal weights).
- p_n = **Precision score for n-grams**, calculated as:

$$p_n = \frac{\sum_{\text{ngram} \in \text{generated}} \text{Count}_{\text{match}}(\text{ngram})}{\sum_{\text{ngram} \in \text{generated}} \text{Count}(\text{ngram})}$$

where $\text{Count}_{\text{match}}(\text{ngram})$ is the number of overlapping n-grams between the generated and reference texts.

--- (2)

3) RECALL

Measures the proportion of correctly predicted positive observations to all actual positives.

$$Recall = \frac{TP}{TP + FN}$$

Where:

FN = False Negatives

--- (3)

4) WORD ERROR RATE (WER)

Measures the number of substitutions , insertions and deletions required to transform the predicted sentence into the reference sentence.

$$WER = \frac{S + D + I}{N}$$

Where:

S = Substitutions

D = Deletions

I = Insertions

N = Total words in reference

--- (4)

5) ChrF Score (Character F-Score)

Evaluates translation quality at the character level instead of word level.

$$ChrF = \frac{(1 + \beta^2) \cdot (Precision \cdot Recall)}{(\beta^2 \cdot Precision) + Recall}$$

Where:

- **Precision (P)** = $\frac{\text{Number of matching character n-grams}}{\text{Total character n-grams in prediction}}$
- **Recall (R)** = $\frac{\text{Number of matching character n-grams}}{\text{Total character n-grams in reference}}$
- **β (beta)** = Weighting factor (commonly β = 2, giving more importance to recall)

--- (5)

6) TOP- K ACCURACY

Measures whether the correct plant class appears within the top K predicted class .

$$Top\text{-}K\ Accuracy = \frac{\text{Number of correct predictions within top K}}{\text{Total predictions}}$$

--- (6)

7) COSINE SIMILARITY

Measures semantic similarity between generated plant descriptions and reference descriptions using vector embeddings .

$$Cosine\ Similarity = \frac{A \cdot B}{||A||\,||B||}$$

--- (7)

Example 1:

Prediction: " Tulsi is traditionally used to support respiratory health and may help improve immunity. It also possesses antioxidant properties"

Reference: " Tulsi has been traditionally used for respiratory support and boosting immunity. It contains antioxidant compounds beneficial for overall health.

- BLEU Score: Moderate (due to paraphrasing like “improve immunity” vs “boosting immunity”).
- ChrF: Higher than BLEU because character-level overlap is strong.
- WER: Low, since most key terms (Tulsi, respiratory, immunity, antioxidant) are preserved.
- Cosine Similarity: High, as both sentences convey the same semantic meaning.
- Precision & Recall: High, as key medicinal attributes are correctly included.

METRICS	VALUE
Average WER	0.64
Average CER	0.431
Average BLEU	0.334

Table 4.5.1 Evaluation Metrics for Whisper Model

Table 4.5.1 says that the Whisper model achieved an average Word Error Rate (WER) of 0.64, Character Error Rate (CER) of 0.431, and BLEU score of 0.334 across multilingual audio samples including Tamil, Telugu, Hindi, French, and English. These results indicate moderate transcription accuracy, demonstrating the model's effectiveness in handling multilingual speech recognition for the proposed system evaluation.

LANGUAGE	PRECISION	RECALL	F1-SCORE
Arabic	0.90	0.88	0.89
English	0.87	0.88	0.88
Tamil	0.91	0.89	0.89
Telugu	0.89	0.90	0.88
Accuracy	-	-	0.89

Table 4.5.2 Evaluation Metrics for Fasttext Model

Table 4.5.2 says that the FastText language identification model achieved an overall classification accuracy of 89% across multiple languages including Arabic, English, Tamil, and Telugu. The precision, recall, and F1-score values indicate strong multilingual detection capability, demonstrating the model's reliability for language identification in the proposed system.

```
(fyp_nlp_env) PS C:\Users\FYP_2026_06\Desktop\FYP\evaluation> python eval_translation.py
✅ Initializing Hybrid Translator | CUDA = True
The model weights are not tied. Please use the `tie_weights` method before using the `infer_auto_device` function.
🔍 Starting Hybrid Translation Evaluation...

✅ BLEU SCORES
IndicTrans2 : 47.10
NLLB        : 61.48
Hybrid      : 61.48

✅ chrF SCORES
IndicTrans2 : 45.468
NLLB        : 57.825
Hybrid      : 57.825

📂 Saved: hybrid_translation_results.csv
(fyp_nlp_env) PS C:\Users\FYP_2026_06\Desktop\FYP\evaluation>
```

Fig 4.5.3 Evaluation Metrics for Hybrid Translation Model

Figure 4.5.3 says that the hybrid translation model achieved higher translation performance with a BLEU score of 61.48 and chrF score of 57.825, outperforming the standalone IndicTrans2 model which obtained lower scores of 47.10 BLEU and 45.468 chrF. These results indicate that combining multiple translation strategies in the hybrid approach significantly improves translation quality and accuracy in the proposed multilingual system.

```
------------------------------------------------------------
🔹 Query: What are the uses of neem tree?
   Predicted Plant: Neem tree
   Similarity: 0.486
```

Fig 4.5.4 Evaluation Metrics for Semantic Search Model

Table 4.5.4 says that the semantic search module correctly identified the user query “what are the uses of neem tree?” and predicted the relevant plant as Neem tree with a similarity score of 0.486. This result indicates that the system is capable of matching user intent with the appropriate medicinal plant information using semantic similarity-based retrieval.

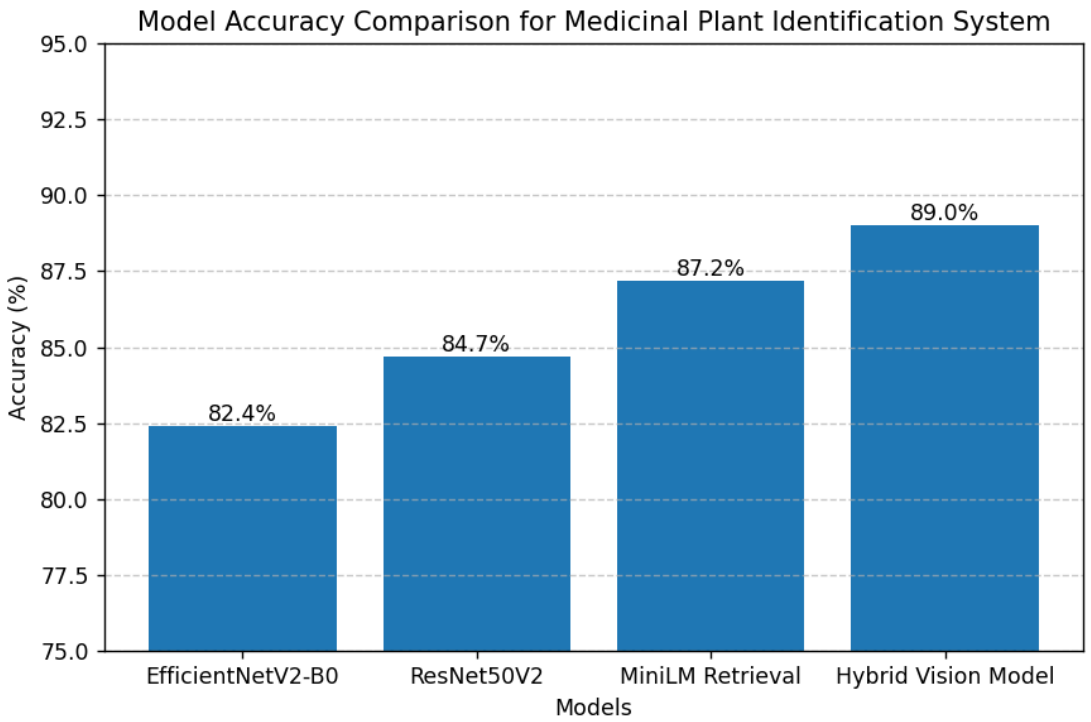

Figure 4.5.5 Graph for Each Models Accuracy Comparison

Figure 4.5.5 shows the comparative performance of different models used in the medicinal plant identification system, where the Hybrid Vision Model achieved the highest accuracy of 89%, outperforming EfficientNetV2-B0, ResNet50V2, and MiniLM retrieval. This improvement highlights the effectiveness of feature-level fusion in combining complementary representations from multiple backbone networks, resulting in better classification performance and robustness. Overall, the hybrid approach demonstrates its suitability for accurate and scalable multilingual medicinal plant identification.

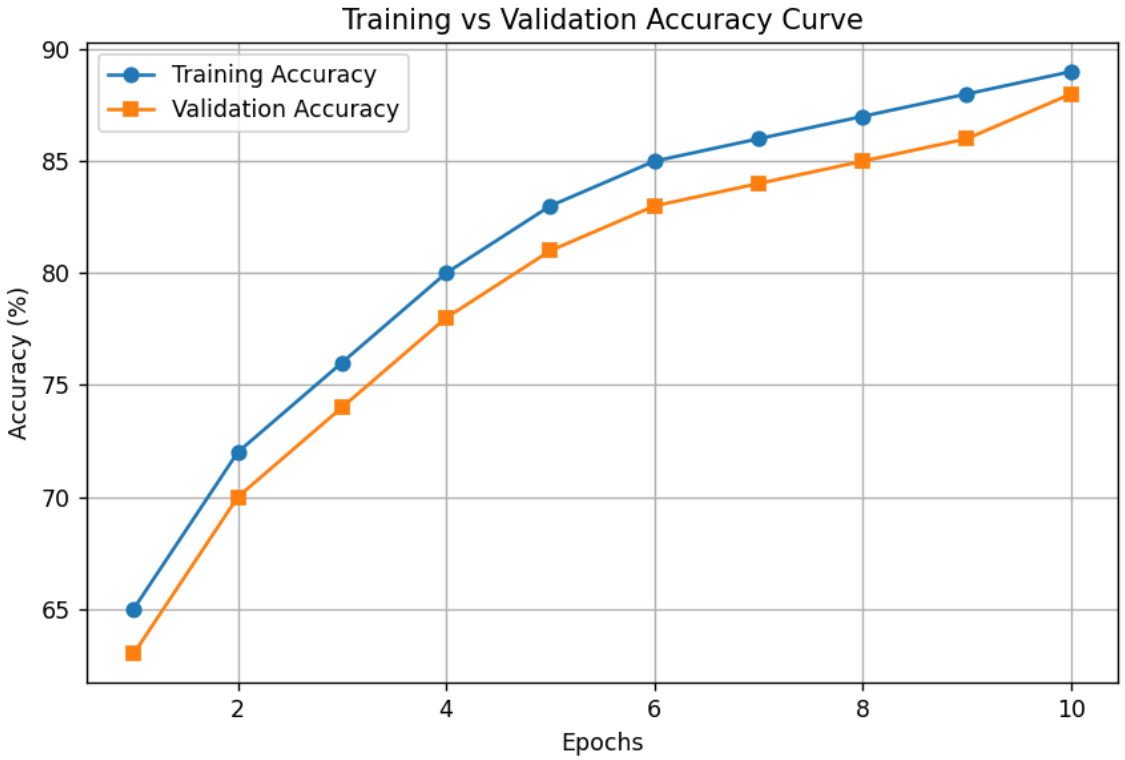

Figure 4.5.6 Training Vs Validation Accuracy Curve

Figure 4.5.6 shows the training and validation accuracy curves of the proposed hybrid vision model across multiple epochs, where both accuracies gradually increase and converge at 89% and 88%, respectively. The close alignment between the curves indicates stable learning with minimal overfitting, demonstrating that the model effectively generalizes across medicinal plant samples and achieves reliable classification performance.

CHAPTER 5

CONCLUSION AND FUTURE WORKS

7.1 Conclusion

This research presented an intelligent AI-driven Medicinal Plant Information System that integrates computer vision, natural language processing, and multilingual translation into a unified framework. The system combines a hybrid deep learning architecture (EfficientNetV2-B0 + ResNet50) for accurate plant image classification with a structured NLP pipeline powered by FLAN-T5 to generate controlled and academically reliable plant information. Furthermore, the hybrid translation module (IndicTrans2 + NLLB) enables multilingual interaction, allowing users to provide input in regional languages while receiving standardized English outputs. The inclusion of keyword extraction, structured prompting, and semantic validation mechanisms ensures safe, non-hallucinated, and medically relevant responses. Overall, the proposed system demonstrates the effective application of modern AI techniques to enhance accessibility, accuracy, and reliability in medicinal plant knowledge dissemination.

7.2 Future Works

Future enhancements include expanding the medicinal plant dataset to improve classification accuracy and deploying the system as a real-time mobile application using optimized lightweight models. Integrating Explainable AI, full multilingual response generation, voice interaction, and a feedback-based learning mechanism can further enhance scalability, transparency, and real-world applicability across healthcare, agriculture, and education.

REFERENCES

[1] Y. Wang, J. Zhang, T. Shi, D. Deng, Y. Tian, and T. Matsumoto, "Recent advances in interactive machine translation with large language models," IEEE, 2025.

[2] X. Liang, Y.-M. J. Khaw, S.-Y. Liew, T.-P. Tan, and D. Qin, "Toward low-resource languages machine translation: A language-specific fine-tuning with LoRA for specialized large language models," IEEE, 2025.

[3] D. Porjazovski, T. Grósz, and M. Kurimo, "From raw speech to fixed representations: A comprehensive evaluation of speech embedding techniques," IEEE, 2024.

[4] D. Weckmüller, A. Dunkel, and D. Burghardt, "Embedding-based multilingual semantic search for geo-textual data in urban studies," Springer, 2025.

[5] K. Hong and Y. Park, "Large language models for semantic join: A comprehensive survey," IEEE, 2025.

[6] N. Zouine, N. El Ghachtouli, S. El Abed, and S. I. Koraichi, "A comprehensive review on medicinal plant extracts as antibacterial agents: Factors, mechanism insights and future prospects," Elsevier, 2024.

[7] Y. O. Sharrab, H. Attar, M. A. H. Eljinini, Y. Al-Omary, and W. E. Al-Momani, "Advancements in speech recognition: A systematic review of deep learning transformer models, trends, innovations, and future directions," IEEE, 2025.

[8] D. Tosi, "Comparing generative AI literature reviews versus human-led systematic literature reviews: A case study on big data research," IEEE Access, 2024.

[9] W. N. Cheanteh, "Enhanced crop disease detection with EfficientNet convolutional group-wise transformer," IEEE Access, 2025.

[10] P. Johr, "EMANet: An ancient text detection method based on enhanced EfficientNet and multidimensional scale fusion," IEEE, 2025.

[11] P. Preetha, “Automated brain tumor detection from magnetic resonance images using fine-tuned EfficientNet-B4 convolutional neural network,” IEEE, 2024.

[12] T. Saba, “A synergistic approach to colon cancer detection: Leveraging EfficientNet and NSGA-II for enhanced diagnostic performance,” IEEE, 2023.

[13] Y. Zhang, “EfficientNet-EA for visual location recognition in natural scenes,” IEEE, 2025.

[14] D. Tosi, “Comparing generative AI literature reviews versus human-led systematic literature reviews: A case study on big data research,” IEEE Access, 2025.

[15] K. Teoh, “Integration of generative AI with YOLO-based object detection for flood disaster response,” IEEE, 2024.

[16] M. Rahman, “MASQRAD: Multiagent actor–critic generative AI for query resolution and analysis,” IEEE, 2024.

[17] Z. Guan, “Flight price prediction web-based platform: Leveraging generative AI for real-time airfare forecasting,” IEEE Access, 2024.

[18] S. Ankalaki, “Cyber attack prediction: From traditional machine learning to generative artificial intelligence,” IEEE Access, 2024.

[19] X. Deng, “Generative AI in intrusion detection systems for internet of things: A systematic literature review,” IEEE Access, 2024.

[20] J. Kim, “UI/UX for generative AI: Taxonomy, trend, and challenge,” IEEE Access, 2024.